



华南理工大学
South China University of Technology

Ya2yOS

内核设计文档

参赛队员：饶晓杰

指导老师：杨磊

面向外部人员的实现级设计报告

Rust · RISC-V 64 · LoongArch64

版本：0.5

代码快照：HEAD 9bc59656 (2026-08-11, 含 sigreturn trampoline 调整)；工作树另有 Makefile/.vscode 本地修改

发布日期：2026-08-11

摘要

Ya2yOS 是一个以 Rust 实现、面向 Linux 用户态兼容的实验性操作系统内核，当前支持 RISC-V 64 与 LoongArch64 QEMU 平台。本文档从可复核的实现视角阐述内核的启动与异常入口、地址空间和页面生命周期、进程线程与编译期可选的 CFS/RR 调度、信号、Linux 风格系统调用与 VFS、网络栈和 VirtIO 设备接入，并给出模块边界、关键不变量、验证约定和已知边界。设计描述以当前源代码为准；对未完成能力使用明确的边界表述，避免将规划误作已实现功能。

关键词：操作系统内核；Rust；Linux ABI；CFS；进程调度；虚拟内存；VFS；RISC-V；LoongArch

文档范围 本文面向评审、协作者和后续维护者。其描述对应 HEAD 9bc59656 (2026-08-11, 含 sigreturn trampoline 调整)；工作树另有 Makefile/.vscode 本地修改；源代码变更后，应同步核对受影响章节。测试故障的逐案证据不在本文展开，而位于 Docs/决赛文档/problem/。

版本与阅读约定

项目	说明
文档版本	0.5
适用范围	Ya2yOS 当前工作树的内核实现；不将规划能力视为既有功能
主体源码	os/src/；构建入口为仓库根目录 Makefile
术语约定	代码标识符、Linux ABI 名称和路径保持原文；其他叙述使用中文
可追溯性	各章给出关键目录；末章提供源码—章节索引与外部参考文献

目录

第一章	概述	6
1.1	定位与范围	6
1.2	设计目标	6
1.3	模块组织	6
1.4	启动与用户态主线	7
1.5	用户态接口与对象模型	7
1.6	已实现与限制	8
第二章	启动、架构与异常入口	9
2.1	启动序列	9
2.2	架构抽象	9
2.3	Trap 到用户态返回	9
2.4	信号返回的架构入口	10
2.5	异常边界	10
第三章	进程、线程与程序映像	11
3.1	模块边界与对象模型	11
3.2	调度策略与全局任务表	13
3.3	rseq：线程级可重启序列	14
3.4	创建：clone 与 clone3	14
3.5	映像替换：execve	15
3.6	退出、僵尸与等待	16
3.7	初始进程与当前边界	16
第四章	内存管理	18
4.1	模块边界与基本对象	18
4.2	架构与地址布局	19
4.3	映射区域、页表与物理帧	19
4.4	多核与异构架构下的一致性	20
4.5	ELF、brk 与按需分配	21
4.6	fork 与写时复制	21
4.7	mmap、munmap 与 mprotect	22
4.8	uaccess 设计方案	23
4.9	System V 共享内存与用户复制	23
4.10	当前边界	24
第五章	信号机制	25
5.1	模块与数据模型	25
5.2	发送、权限与唤醒	25
5.3	递送与默认动作	26
5.4	用户信号帧与返回	26
5.5	相关系统调用与限制	27
第六章	网络模块	28
6.1	概述	28
6.2	架构分层	28

	6.3 全局对象	28
	6.4 初始化流程	29
	6.5 路由与设备	29
	6.6 TCP 套接字	29
	6.7 UDP 套接字	30
	6.8 Unix Domain Socket	30
	6.9 Socket Option	31
	6.10 系统调用与 File 统一	31
	6.11 数据收发路径	31
	6.12 当前边界	32
第七章	I/O 设备	33
	7.1 设备驱动架构	33
	7.2 VirtIO 支持	33
	7.3 块设备	34
	7.4 网络设备	34
	7.5 控制台与字符类设备	35
	7.6 LoongArch64 PCI	35
	7.7 中断与轮询策略	35
	7.8 /dev 兼容层	36
	7.9 Loop 设备	36
	7.10 当前边界与后续方向	36
第八章	文件系统	37
	8.1 概述	37
	8.2 VFS 抽象	37
	8.3 文件描述符与进程文件上下文	39
	8.4 路径解析与打开文件	40
	8.5 目录项与 inode 缓存	41
	8.6 文件页缓存	41
	8.7 ext4 与 lwext4 集成	42
	8.8 普通文件 IO	43
	8.9 目录项与元数据	44
	8.10 管道、FIFO 与 splice	44
	8.11 devfs、loop 设备与 proc 动态文件	45
	8.12 IO 多路复用与事件 fd	46
	8.13 挂载接口	46
	8.14 文件系统锁设计	47
	8.15 文件锁、fcntl 与 xattr	48
	8.16 动态链接支持	48
	8.17 当前边界与后续方向	48
第九章	lwext4 与磁盘文件系统引擎	49
	9.1 概述与定位	49
	9.2 构建与绑定	49
	9.3 磁盘布局模型	49
	9.4 块缓存 bcache	50

9.5	日志与事务	51
9.6	文件操作流程	51
9.7	SMP 锁钩子	52
9.8	Ya2yOS 定制与写回缓存	53
9.9	当前边界	53
第十章	AI 的使用情况	54
10.1	使用概况	54
10.2	主要工作成果	54
10.3	使用中的不足	55
10.4	个人反思	55
第十一章	总结与展望	56
11.1	项目成果总结	56
11.2	项目特色	56
11.3	与同类项目的对比	57
11.4	已知局限	57
11.5	未来发展方向	57
11.6	工程实践反思	58
11.7	结语	58
第十二章	实现追溯与参考资料	59
12.1	源码—章节索引	59
12.2	参考资料	59

第一章 概述

1.1 定位与范围

Ya2yOS 是以 Rust 编写的宏内核实验系统，基于 TatlinOS 持续演进，面向 RISC-V64 和 LoongArch64 QEMU 平台提供 Linux 用户态 ABI 的高频兼容路径。内核运行在各架构的内核特权级；进程、内存、VFS、信号、网络 and 驱动位于同一内核映像中，由 Rust 类型、引用计数和显式同步原语约束对象生命周期。

本文档是实现级说明，而非功能承诺。系统调用已经覆盖进程、内存、文件、信号、时间、同步、网络和 I/O 多路复用等类别；各接口是否具有完整 Linux 语义须以对应章节的“当前边界”为准。尤其是挂载、虚拟文件系统、异步 I/O、设备模型和部分低频 syscall 仍以兼容实现为主。

1.2 设计目标

1. **Linux ABI 优先**：通过 Linux syscall 编号、架构 ABI、errno 和用户内存访问规则，使 glibc、musl、BusyBox、libc-test 及 LTP 子集能够沿常见路径运行；
2. **完整核心链路**：覆盖 ELF 装载、clone/execve/wait、页表和 COW、ext4 文件访问、信号递送、socket 与 VirtIO 设备，使用户负载可从启动到退出闭环运行；
3. **双架构复用**：把页表、陷入上下文、时钟和设备传输差异隔离在 arch/ 与 drivers/virtio/，让任务、内存、文件和 syscall 高层逻辑共用；
4. **可验证地演进**：新能力或修复以用户态测例和 QEMU 日志验证；对暂未实现的语义明确标注为限制或 stub，而不把接口存在等同于能力完整。

1.3 模块组织

os/src/main.rs 负责按依赖顺序初始化子系统；arch、drivers 提供平台能力，trap 和 syscall 构成用户态 ABI 入口，其他模块保存核心状态和语义。当前目录边界如下：

```
os/src/
├─ arch/      # RISC-V64 / LoongArch64 上下文、页表、时钟、trap 与平台代码
├─ drivers/   # VirtIO block/net、设备容器及架构相关 transport
├─ trap/      # 用户陷入、页故障、时钟中断与返回用户态
├─ syscall/   # Linux syscall 分发；task/mm/fs/net/signal/sync/io_mpx 等
ABI 入口
├─ task/      # Process、TCB、可选 CFS/RR 调度、futex、clone/exit/wait 支撑
├─ mm/        # MemorySet、VMA、帧分配、ELF、用户复制、COW、共享内存
├─ fs/        # ext4/VFS 适配、fd、pipe、epoll、设备、挂载记录与 proc 兼容
├─ signal/    # action 表、pending、递送、信号帧和 interval timer 信号
├─ net/       # smoltcp service、路由、TCP/UDP/Unix socket 与网络设备包装
├─ timer/     # 时钟、超时、rusage 与时间 ABI 数据类型
├─ sync/      # 内核同步封装
└─ utils/     # errno、ID、poll、资源槽位和通用辅助类型
```

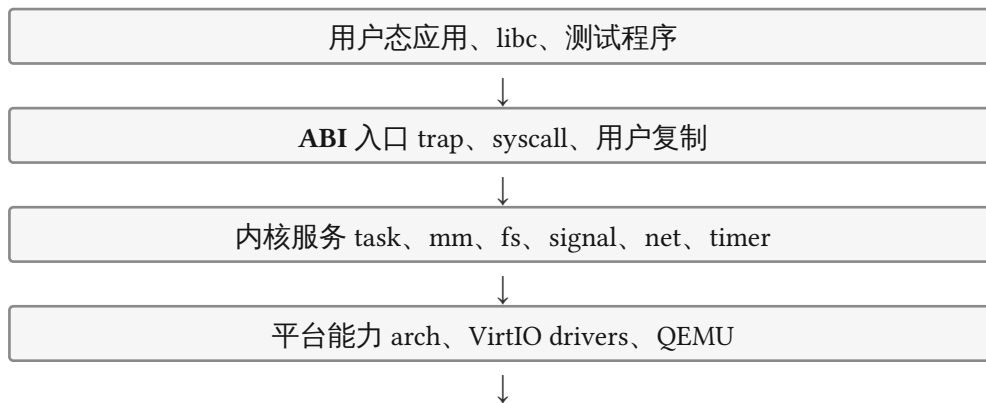


图 1 Ya2yOS 的分层关系：上层经 ABI 使用内核服务，平台差异收敛到架构与驱动层。

1.4 启动与用户态主线

汇编入口位于 `arch/*/qemu/asms/entry.asm`。首个 hart 进入 `rust_main()` 后依次完成 时钟频率、内存、日志、trap、任务、文件系统和网络初始化；随后创建 `/initproc` 对应的 初始 TCB，发布启动屏障并使能定时器。`run_tasks()` 将 runnable 任务交给 `task/scheduler/` 中由 feature 选定的策略，再取出下一任务；默认 CFS 按最小 `vruntime` 选择，RR 配置按 FIFO 轮转。首次运行由 `trap_return` 恢复用户 trap context。

其他 hart 在 `INIT_FINISHED` 前自旋等待，之后安装 trap 向量、激活内核地址空间并设置 定时器。RISC-V64 与 LoongArch64 的 QEMU 配置分别提供 8 个和 12 个 hart；当前代码已 接入 per-Hart processor、远程入队、空闲 hart 通知、IPI 协作和 remote TLB mailbox，但 尚未形成完整的跨 hart 迁移、work stealing 或负载均衡策略。网络由 net feature 控制，默认启用：RISC-V 在未发现 VirtIO-net 时仍保留 loopback，LoongArch 通过 PCI 路径建立 设备 transport。

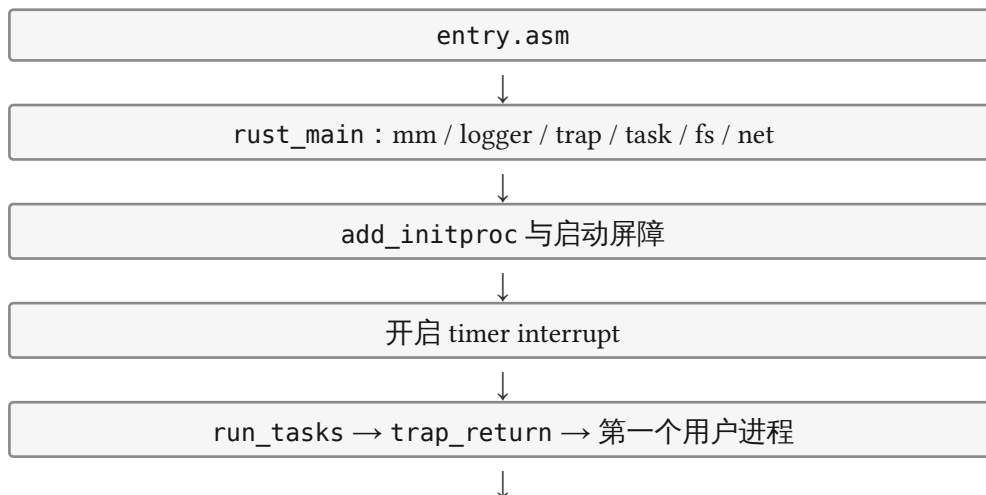


图 1 从平台入口到第一个用户进程的当前启动主线。

1.5 用户态接口与对象模型

系统调用号在 `syscall::Syscall` 中定义，以 `num_enum` 将数字转换为枚举并在 `syscall()` 中分发。trap 层从用户寄存器读取 syscall 号和六个参数，处理函数通过 `copy_from_user`、`copy_to_user` 及其 typed wrapper 访问用户内存；`SysErrNo` 最终由 trap 层转换为用户可见的负 `errno` 返回值。

系统中最重要的对象关系为：Process 保存线程组资源（地址空间、fd 表、文件系统上下文和信号动作表），TaskControlBlock 保存一个可调度线程的 trap context、内核栈、线程级信号与等待

状态；MemorySet 管理该进程的页表和 VMA；FileDescriptor 统一持有普通文件、管道、socket、事件对象和挂载上下文 fd。

Process MemorySet、 FdTable、FSInfo、 SigTable	TCB 线程上下文、内 核栈、pending signal、 futex	FileDescriptor file、 pipe、socket、event、 mount fd	MemorySet 页表、 VMA、frame 引用
--	---	---	--------------------------------------

图 1 进程资源、线程执行状态、文件描述符和地址空间的主要归属关系。

1.6 已实现与限制

内核已具备 ext4 后端、ELF 动态解释器映射、COW、文件 mmap、System V 共享内存、POSIX 风格信号、TCP/UDP/Unix socket、pipe/eventfd/epoll、VirtIO block/net 与 RISC-V MMIO、LoongArch PCI 传输等主线能力。挂载记录同时支持叠加、bind/move 子树和 shared/slave 传播状态，但路径解析仍使用底层 ext4 目录；它不是完整的 VFS 挂载树或 mount namespace 实现。其余限制分别在第 4 至第 9 章中说明。

实现追溯：os/src/main.rs、os/src/syscall/mod.rs、os/src/task/、os/src/mm/、os/src/fs/、os/src/signal/、os/src/net/

第二章 启动、架构与异常入口

2.1 启动序列

汇编入口位于各架构的 `arch/*/qemu/asms/entry.asm`。RISC-V 的 trampoline 完成早期地址偏移处理后进入 `rust_main(hartid)`；LoongArch 使用对应的平台入口。首个 hart 清零 BSS，初始化时钟频率，再按顺序建立核心运行环境。

实现追溯：`os/src/main.rs`、`os/src/arch/*/qemu/asms/entry.asm`

`entry.asm` → `rust_main` → `mm::init` → `logger::init` → `trap::init` → `task::init` → `fs::init` → `net::init_network` → `add_initproc` → 开启时钟中断 → `run_tasks`

非首 hart 等待 `INIT_FINISHED`，随后各自安装 trap 向量、激活内核页表、开启定时器并进入调度循环。`START_HART_ID` 用于区分负责列举应用和启动初始调度的 hart；RISC-V64 与 LoongArch64 的 QEMU 配置分别声明 8 个和 12 个 hart。当前已存在远程入队、空闲 hart 通知和 IPI 路径，但调度迁移与负载均衡仍不完整。

2.2 架构抽象

`os/src/arch/mod.rs` 是与上层交互的汇聚点。两个架构分别提供：

- 任务和 trap 寄存器上下文，以及 `switch.S` 完成的上下文切换；
- 页表建立、地址翻译和 TLB 刷新；
- 时钟频率初始化、下一次时钟触发和中断开关；
- UART 控制台、链接脚本、物理内存布局和启动汇编；
- RISC-V 的 SBI 路径，以及 LoongArch 的 PCI/VirtIO 平台接入。

架构相关代码应保持在 `arch/` 或驱动的架构子目录中。上层不直接写 CSR 或平台寄存器，而经 `trap_interface`、`time`、`cpu` 和页表接口调用。

2.3 Trap 到用户态返回

`trap::trap_handler()` 是用户态进入内核的统一门。它从当前线程取得 trap context，根据异常原因选择系统调用、页故障、定时器中断或其他异常处理，并在准备恢复用户寄存器前检查待处理信号。

事件	主要处理	可见结果
用户 <code>ecall</code>	递增返回地址，读取 <code>syscall</code> 号和 6 个参数，调用 <code>syscall()</code>	返回值写回用户寄存器
load/store/instruction page fault	查询 <code>MemorySet</code> 的 VMA，执行懒分配、COW 或文件页处理	成功后重试，非法访问转换为异常信号
timer interrupt	更新时间、设置下一次触发，驱动抢占/唤醒相关路径	可切换到其他 ready 任务
外部中断	经架构中断接口和设备驱动分发	设备状态推进或唤醒等待者

异常路径不能信任用户提供的地址、长度或结构体。所有跨地址空间读写使用内存模块的受检辅助函数，错误以 Linux `errno` 或适当信号反馈给用户程序。

2.4 信号返回的架构入口

信号 handler 的返回地址由 `setup_frame()` 写入 `libc restorer` 或架构相关的 `sigreturn_trampoline`。用户 handler 返回后, `trampoline` 进入 `rt_sigreturn`, 由 `restore_frame()` 通过用户地址访问辅助函数读取信号帧, 恢复架构 `trap context`、`signal mask`、备用栈状态和原始返回值, 再经统一的用户态返回路径继续执行。高层协议在两种架构间共用, 但 `machine/trap context` 布局和汇编入口分别由 `arch/riscv64/` 与 `arch/loongarch64/` 提供; 因此修改 `trampoline` 时必须同时检查信号帧布局和 `trap.S`。

2.5 异常边界

异常路径对用户地址、长度和结构体执行受检访问; 可修复的页故障进入懒分配、文件映射或 COW, 无法修复的访问转换为 `SIGSEGV/SIGBUS`。架构 `IRQ` 抽象仍有未完成的硬件 `acknowledge/enable` 路径, 部分未支持 `trap` 和 `timer condvar` 分支仍可能 `panic`, 不能将统一 `trap` 入口表述为所有硬件异常均已实现。

RISC-V 的特权级、异常和地址转换语义以《The RISC-V Instruction Set Manual, Volume II: Privileged Architecture》为准; LoongArch 的平台差异遵循《LoongArch Architecture Reference Manual, Volume 1: Basic Architecture》。完整报告的参考资料统一列于 `main.typ` 末尾。

第三章 进程、线程与程序映像

本章对应 `os/src/task/`、`os/src/syscall/task/` 与 `os/src/mm/memory_set/elf_loader.rs` 在当前工作树中的实现。它说明已经接入的 Linux 风格任务模型、创建和回收路径；调度策略、资源回收和部分 `clone3` 字段仍有明确限制，文中据实标出而不将其表述为规划中的完整能力。

3.1 模块边界与对象模型

位置	当前职责
<code>os/src/task/process/process.rs</code>	Process、全局 PID 映射、父子关系、进程组/会话和线程组退出元数据
<code>os/src/task/task/task.rs</code>	TaskControlBlock、用户 trap 上下文、内核栈，以及 <code>new</code> 、 <code>clone_process</code> 、 <code>exec</code>
<code>os/src/task/manager.rs</code>	TID 到 TCB 的映射、futex 唤醒和阻塞任务计时器补扫
<code>os/src/task/scheduler/</code>	编译期选择的 CFS/RR 策略、就绪队列门面与运行时间记账
<code>os/src/task/processor.rs</code>	按 Hart 保存当前任务和 idle 上下文，执行调度循环
<code>os/src/task/mod.rs</code>	<code>suspend</code> 、 <code>block</code> 、 <code>stop</code> 、 <code>exit</code> 和 <code>initproc</code> 的控制流入口
<code>os/src/syscall/task/</code>	<code>clone</code> 、 <code>clone3</code> 、 <code>execve</code> 、 <code>exit</code> 、 <code>waitpid</code> 与 <code>waitid</code> 的 ABI 入口

Ya2yOS 将可调度执行单元与进程级资源分开建模。TaskControlBlock (TCB) 代表一个线程，持有独立的 `TidHandle`、`KernelStackOnHeap`、用户 trap 上下文、内核切换上下文和线程私有的信号、futex、凭据、计时器等状态。Process 代表线程组和其进程级资源，持有当前地址空间、信号动作表、文件描述符表、文件系统上下文及 PID。TCB 通过 `Arc<Process>` 归属到一个 Process；同一线程组内的多个 TCB 因而观察同一组进程资源。

```
// 代码块只摘录影响本章流程的字段；源码还包含调度、资源限制等字段。
pub struct Process {
    memory_set: ResourceSlot<MemorySet>,
    sig_table: ResourceSlot<Mutex<SigTable>>,
    pub fd_table: Arc<FdTable>,
    pub fs_info: Arc<FSInfo>,
    pub pid: usize,
    home_hart: usize,
    pub meta: Mutex<ProcessMeta>,
    pub rlimit_fsize: Mutex<RLimit>,
}

pub struct TaskControlBlock {
    tid: TidHandle,
    kernel_stack: KernelStackOnHeap,
    pub process: Arc<Process>,
    cpu_affinity: AtomicUsize,
    scheduled_hart: AtomicUsize,
    on_cpu: AtomicBool,
    pub interrupted: AtomicBool,
```

```

pub interrupt_waker: AtomicWaker,
pub(crate) sched_entity: SchedEntity,
inner: RemoteTlbMutex<TaskControlBlockInner>,
}

```

ResourceSlot<MemorySet> 和 ResourceSlot<Mutex<SigTable>> 的作用是保护 Arc 指针整体的读取和替换。地址空间本身由 MemorySet 内部同步；FdTable 和 FSInfo 也各自管理内部并发。因此，execve 替换映像时可调用 Process::change_memory_set_and_sigtable() 更换地址空间和信号动作表，而普通文件操作不需要持有一个覆盖整个 Process 的大锁。

ProcessMeta 保存不属于资源槽位的线程组关系和可等待状态：活线程弱引用列表、子进程弱引用列表、父 PID、is_child_subreaper、进程组 ID (PGID)、会话 ID (SID)、子进程事件 AtomicWaker、退出信号、线程组退出码、stop/continue 事件、终止信号、资源使用快照、comm 和 personality。全局 PID_2_PROCESS_ARC 是 BTreeMap<usize, Arc<Process>>；进程创建时插入，父进程实际回收 zombie 时才由 Process::remove_from_global_map() 删除。tasks 与 children 都是弱引用列表，退出或等待路径会清理其中已经失效的引用。

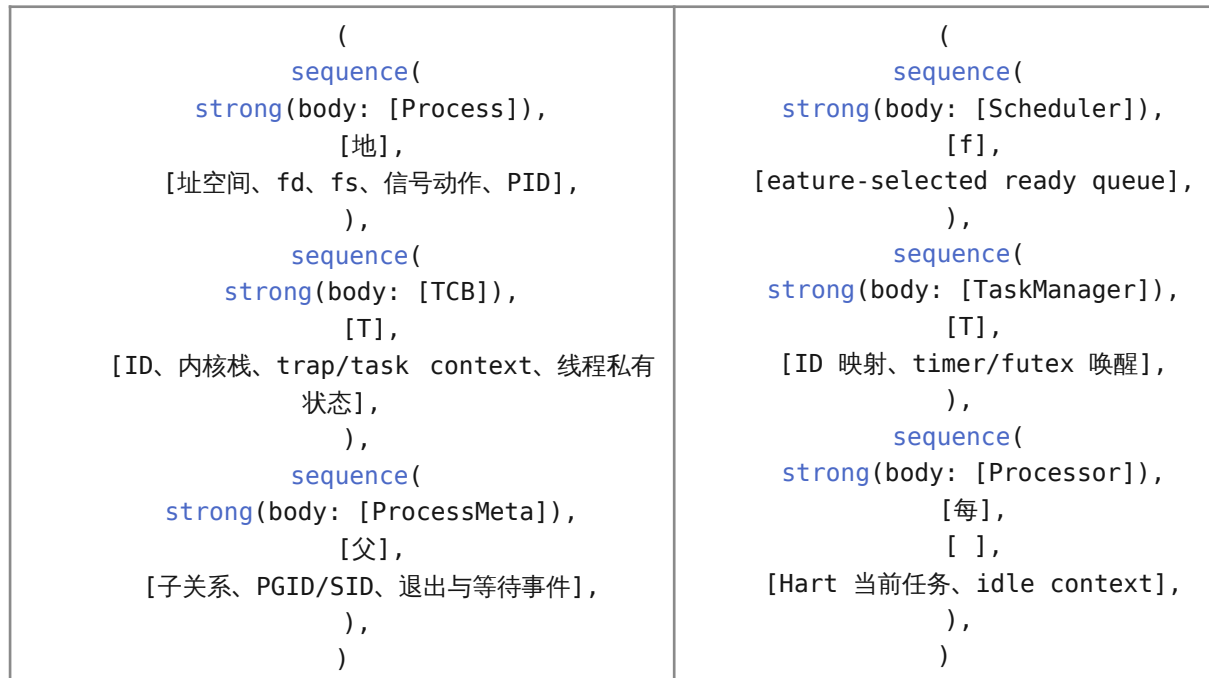


图 2 当前任务管理对象关系。

3.1.1 任务状态与生命周期

TaskStatus 当前定义六种状态：

状态	实现含义
Ready	已可运行，可进入当前编译策略的就绪队列
Running	被当前 Hart 的 Processor.current 持有并正在执行
Blocked	等待 futex、I/O、异步事件等；唤醒路径将其改回 Ready
Stopped	被 stop 类信号停止，等待 SIGCONT 等恢复路径
VforkBlocked	父线程因 CLONE_VFORK 等待指定子线程 exec 或退出
Zombie	当前线程已退出；进程是否可由父进程回收取决于全部线程是否退出

TCB 的 trap 上下文实际位于用户地址空间的专用映射页，TaskControlBlockInner 保存其物理页号和起始虚拟地址；TaskContext 保存内核态切换所需的 callee-saved 寄存器。新任务的 TaskContext::goto_trap_return() 将返回地址设为 trap_return，首次被调度后从 trap 上下文恢复用户寄存器并回到用户态。

TidHandle 使用一个从 1 起始的全局 IdAllocator，initproc 因而获得 PID/TID 1。当前 TidHandle::Drop 不归还 ID：注释明确指出共享的 PID/TID 分配器若在 zombie 被 wait 前重用 ID，会覆盖全局 PID 映射。因此本快照中 TID/PID 单调消耗，不应把它描述为已实现的 ID 回收机制。

3.2 调度策略与全局任务表

task::ready_queue 是稳定门面，具体实现由互斥的 scheduler-cfs 与 scheduler-rr Cargo feature 在编译期选择。默认 scheduler-cfs 使用所有 Hart 共享的 Mutex<CfsRunQueue>，内部 BinaryHeap<CfsEntry> 以 (vruntime, tid) 反向排序，使 pop() 取得虚拟运行时间最小的任务。fetch_task(hartid) 会保留暂时不满足当前 Hart CPU affinity 的堆项，并从同一共享堆中选择可运行任务；因此不存在每 Hart 独立的 CFS heap 或 min_vruntime。TCB 的 SchedEntity 保存 vruntime、exec_start 与原子 on_rq 去重位；任务离开 Processor.current 时按硬件 tick 统计执行时间，并使用 Linux nice -20..19 的权重表折算 vruntime。共享队列的 min_vruntime 单调推进，新任务首次入队时被放置到这一全局坐标。

scheduler-rr 保留原有 FIFO 语义：单个全局 VecDeque 从队尾入队、队首出队，辅以 TID 集合去重，并在取任务时跳过不属于当前 scheduled_hart 的项。CFS 与 RR 都通过同一 add_task() / fetch_task() API 服务唤醒路径。tid_to_task 独立维护 BTreeMap<usize, Arc<TaskControlBlock>>，用于按 TID 查找线程、遍历计时器候选和在线程退出的 idle 控制流中移除条目。

每个 Hart 有一个 Processor，其中包含 current: Option<Arc<TaskControlBlock>> 和 idle_task_cx。run_tasks() 检查普通计时器、阻塞任务计时器和 futex 超时，记账并取出当前任务；仅将 Ready/Running 实体重新入队，再由所选策略统一选择下一个任务并通过 switch() 进入其 TaskContext。Blocked、VforkBlocked 与 Stopped 留在队列外，直到相应唤醒路径把它们恢复为 Ready。

时钟中断仍以 100Hz 调用 suspend_current_and_run_next() 驱动抢占，自愿 yield 和阻塞路径也可切回调度循环。因此这是基于 nice 加权 vruntime 的简化 CFS，而非 Linux 完整调度子系统：尚无 target latency/sched period、调度组、完整的跨 Hart 迁移、work stealing 或负载均衡策略，也没有实现实时调度类。已有远程入队和空闲 Hart 通知，用于唤醒协作而非完整负载均衡。feature 只选择内核内部 runqueue；sched_setscheduler(2) 等用户 ABI 仍是兼容 stub，不提供运行时 CFS/RR 切换。

发起者	关键交互	处理对象
用户态 / 时钟中断	trap 后调用 suspend、block、stop 或 exit	任务控制流
任务控制流	状态更新，必要时回到 scheduler 或等待队列	Scheduler
Scheduler	按 CFS/RR 策略取下一个 Ready 任务	Processor
Processor	switch 到任务 TaskContext	任务控制流

图 3 调度入口与状态转换的当前控制流。

3.3 rseq：线程级可重启序列

Ya2yOS 在 `os/src/task/rseq.rs` 实现 classic 32 字节 Linux `rseq(2)` ABI。注册状态保存在 TCB 的 `TaskControlBlockInner.rseq` 中，因此它属于线程而不是 Process；`abi_addr` 必须按 32 字节对齐，长度必须为 32，内核会先探测并初始化用户 ABI 区域。当前实现发布当前 Hart 的 CPU 编号；由于尚无 NUMA 拓扑或 per-mm 并发 ID，`node_id` 和 `mm_cid` 写为 0，未通过 `auxv` 宣布更新的扩展布局。

注册时只接受 `flags == 0`；重复注册按 ABI 参数返回 `EBUSY` 或参数错误。注销使用 `RSEQ_FLAG_UNREGISTER`，要求地址、长度和签名与当前注册完全匹配，否则返回 `EINVAL` 或 `EPERM`。线程执行 `execve` 时清除旧映像中的 `rseq` 注册，`CLONE_VM` 创建的子线程也清除 `rseq`，以避免继承指向旧线程局部区域的用户指针；普通 `fork` 路径则继承该状态。

在返回用户态前，内核将当前 Hart 写入 `cpu_id_start/cpu_id`，检查用户的 `rseq_cs` 描述符和 `abort` 签名：若指令指针仍位于可重启临界区内，就清除 `rseq_cs` 并把 `trap` 返回地址改为 `abort_ip`；否则仅清除已完成的描述符。用户内存或描述符无效时停止继续尝试，并按错误路径向线程交付 `SIGSEGV`。这使 `rseq` 的 CPU 发布和临界区回滚与 TCB 调度、`exec`、`clone` 生命周期保持一致。

3.4 创建：clone 与 clone3

`sys_clone()` 先解析低位退出信号和 `CloneFlags`，校验不支持或互斥的组合，再用 `TaskControlBlock::clone_process()`；成功后将新 TCB 加入 `ready_queue` 并返回其 TID。`clone_process()` 先分配 TID 和四页内核栈，随后采用两个阶段：第一阶段读取父 TCB/Process 的状态并决定共享或复制策略，第二阶段在不持有父任务锁时构造子 TCB、建立 `trap` 上下文、登记进程任务表和全局 TID 表。这样避免了创建过程中嵌套持有父子任务锁。

标志或情形	当前实现
<code>CLONE_VM</code>	共享父 <code>MemorySet</code> ，但不必然共享 <code>Process</code> ：与 <code>CLONE_THREAD</code> 一起使用时复用父线程组；非线程型 <code>CLONE_VM</code> 子进程创建新的 <code>Process/PID</code> ，但引用同一地址空间。非 <code>CLONE_VM</code> 路径复制用户地址空间。 <code>CLONE_VM</code> 子任务通常清空 <code>alternate signal stack</code> 并清除 <code>rseq</code> ； <code>CLONE_VM CLONE_VFORK</code> 是例外。
<code>CLONE_FS</code>	共享 <code>FSInfo</code> ；否则以 <code>FSInfo::from_another()</code> 复制。
<code>CLONE_FILES</code>	共享 <code>FdTable</code> ；否则以 <code>FdTable::from_another()</code> 复制。
<code>CLONE_SIGHAND</code>	共享信号动作表；否则复制， <code>CLONE_CLEAR_SIGHAND</code> 则创建空表。
<code>CLONE_THREAD</code>	复用父 <code>Process</code> 、 <code>PID</code> 、父 <code>PID</code> 与计时器；非线程创建新的 <code>Process</code> ，其 <code>PID</code> 为新 TID，并登记为父进程子项。
<code>CLONE_SETTLS</code>	把 <code>tls</code> 写入子 <code>trap</code> 上下文的线程指针寄存器。
<code>CLONE_PARENT_SETTID</code> <code>CLONE_CHILD_SETTID</code>	/ 分别向父/子地址空间的用户指针写入新 TID。
<code>CLONE_CHILD_CLEARTID</code>	记录用户指针，线程退出或 <code>exec</code> 替换旧地址空间前写零并 <code>futex</code> 唤醒。
<code>CLONE_VFORK</code>	父 TCB 记录子 TID 并转为 <code>VforkBlocked</code> ；子线程 <code>exec</code> 或退出时唤醒相应父线程。

子 trap 上下文从父线程复制，子路径把返回寄存器 a0 设为 0；用户指定非零栈时再覆盖 sp。非 CLONE_THREAD 的普通子进程会继承父 PGID/SID、设置退出信号，并创建 /proc/<pid> 目录项。CLONE_THREAD 不创建新的 /proc 进程目录。

sys_clone3() 已接入 clone_args 的用户内存读取和版本长度检查，但它是一个适配层：将可支持字段转换为 legacy sys_clone() 参数。set_tid/set_tid_size 非零会直接返回 EINVAL；pidfd 非零时先执行用户指针可读性检查，但 pidfd 功能本身仍因 CLONE_PIDFD 校验失败而不可用；CLONE_INT0_CGROUP 等 cgroup 请求也会被 flags 校验拒绝。因此不能将 clone3 记为完整实现。



图 2 clone 创建路径。

3.5 映像替换：execve

sys_execve() 从当前地址空间复制路径、argv 与 envp。空 envp 采用内核提供的竞赛运行环境；路径按当前 FSInfo.cwd 归一化，/proc/self/exe 被解析为当前可执行文件。入口检查可执行权限和写打开冲突，然后读取 ELF 所需的头、程序头和可加载字节。

若目标不是 ELF，入口会解析 shebang：解释器及其可选参数被重建到 argv 前部，脚本的绝对路径成为解释器参数；/bin/sh 与 /bin/busybox 按镜像兼容规则转到 /musl/busybox。无 ELF 且无有效 shebang 返回 ENOEXEC。历史 .sh 路径也保留了 busybox 兼容分支。

MemorySetInner::from_elf() 新建含内核映射的地址空间，映射所有 PT_LOAD 段，并按 ELF 标志设置用户 R/W/X 权限。若存在 PT_INTERP，内核还会在固定偏移映射动态解释器，并把用户入口设为解释器入口；动态链接和共享库重定位仍由用户态解释器完成。加载器在 ELF 末尾预留 guard page 后建立初始 brk 区域，并生成 AT_PHDR、AT_PHENT、AT_PHNUM、AT_ENTRY、AT_BASE、AT_PAGESZ 等 auxv 项。

TaskControlBlock::exec() 在切换地址空间前处理 clear_child_tid，以保证该用户地址仍属于旧映像；之后替换 MemorySet 和 SigTable，重新分配用户栈/trap 资源，关闭 close_on_exec 文件描述符，清空线程的 pending/mask 信号状态，并在新用户栈写入字符串、指针数组、auxv、随机区与 argc。它还将当前线程的 comm 更新为 argv[0] 的末级名称（最多 16 个字符），并在 CLONE_VFORK 场景恢复等待的父线程。

发起者	关键交互	处理对象
用户态	路径、argv、envp	sys_execve
sys_execve	检查文件、shebang 与 ELF 映像	VFS / ELF loader
TaskControlBlock	替换映像、重建栈并返回用户态	新程序

图 4 execve 当前实现路径。

3.6 退出、僵尸与等待

`sys_exit()` 进入 `exit_current_and_run_next()`，仅终止当前线程；`sys_exit_group()` 先通过 `exit_current_group_and_run_next()` 设置一次线程组退出码，向其他线程组成员发送 `SIGKILL`，并唤醒其中的阻塞任务，然后使每个线程最终走自己的退出路径。`suspend_current_and_run_next()` 也会检测线程组已经退出，防止成员在时钟切换后继续执行。

当前线程退出时，内核依次执行 `CLONE_CHILD_CLEARTID` 写零和 `futex` 唤醒、robust `futex` 死者处理、`vfork` 父线程唤醒、`trap context` 映射移除和 `Zombie` 标记；随后从所属 `ProcessMeta.tasks` 移除自身的弱引用。它还会唤醒阻塞的兄弟线程，并从相关 `futex` 等待队列删除条目，避免后续 `futex` 唤醒或超时把已经清理的线程再次入队。

当最后一个线程退出时，退出路径冻结使用量，清理 `trap area`、地址空间数据、文件描述符与文件系统上下文、`POSIX` 文件锁和文件租约，调用 `exit_and_reparent()` 将仍存活的子进程优先改挂到父系中最近的存活 `child subreaper`，没有时才回退到 `PID 1`，并按子进程的 `exit_signal` 通知父进程和唤醒 `child_exit_event`。已经是 `zombie` 的被收养子进程也会通知新的父进程。`Process` 本身在此时仍保留在全局 `PID` 映射中，以供父进程观察 `zombie`；真正删除发生在成功的等待调用中。

`sys_waitpid()` 支持 `POSIX` `PID` 选择语义（指定 `PID`、当前/指定进程组、任意子进程），并处理 `WNOHANG`、`WNOEXIT`、`WUNTRACED`/`WSTOPPED`、`__WALL` 与 `__WCLONE`。`sys_waitid()` 使用相同的子进程筛选和等待机制，额外报告 `WEXITED`、`WSTOPPED`、`WCONTINUED`，以 `siginfo_t` 返回可见状态。无匹配子进程返回 `ECHILD`；允许阻塞时，调用者在 `ProcessMeta.child_exit_event` 注册 `waker`，退出、`stop` 或 `continue` 事件会令其重新扫描。消费退出事件且未指定 `WNOEXIT` 时，父进程累计子进程的资源使用量、从 `children` 删除弱引用，再调用 `Process::remove_from_global_map()` 回收进程记录。

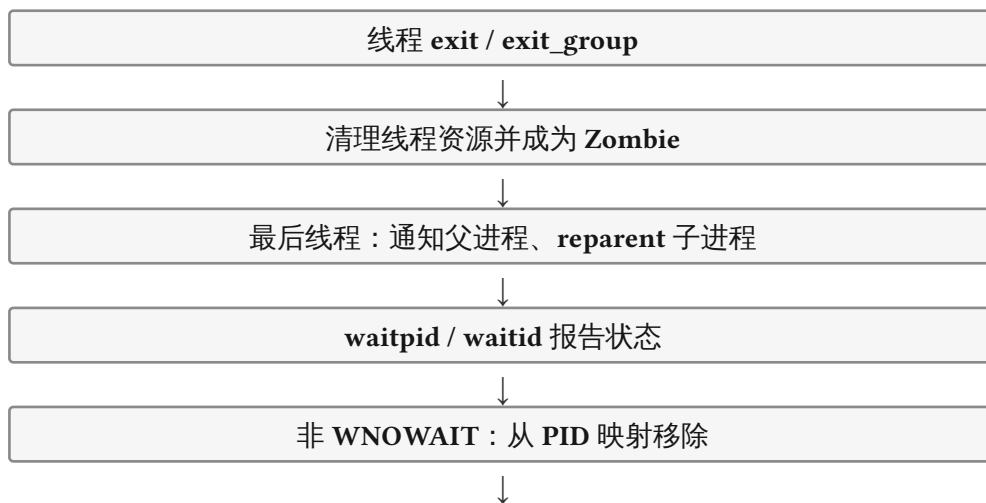


图 3 退出到父进程回收的生命周期。

3.7 初始进程与当前边界

`INITPROC` 是惰性初始化的 `Arc<TaskControlBlock>`。首次访问时，内核从根文件系统打开 `/initproc`，读取 `ELF` 并调用 `TaskControlBlock::new()`；该函数创建 `PID/TID 1` 的 `Process`、带标准输入输出的 `FdTable`、初始 `FSInfo`、用户地址空间、四页内核栈及初始 `trap context`。`add_initproc()` 把它放入当前调度策略的就绪队列和 `TID` 映射，之后 `run_tasks()` 才能首次进入用户态。孤儿进程会在父进程退出时重新挂到 `PID 1`。

边界	当前状态
调度策略	默认启用简化 CFS ; make SCHEDULER=rr 编译 FIFO RR。两个 feature 互斥，不能在运行时切换。
ID 生命周期	TidHandle 当前不归还 ID ,避免 zombie 等待期间发生 PID/TID 重用。
clone3	仅将支持的 clone_args 字段转换到 sys_clone ;set_tid、pidfd 和 cgroup 等扩展未实现。
ELF 动态加载	内核映射 PT_INTERP 指定的解释器并提供 auxv ;共享库解析与重定位在用户态完成。
Linux 调度 ABI	sched_setscheduler/sched_getscheduler 等仍为兼容实现，不代表完整 SCHED_OTHER/实时类语义。
多核	CFS 使用所有 Hart 共享的就绪堆，并按线程 CPU affinity 过滤当前 Hart 不可运行的任务；RR 使用全局 FIFO 队列并按 scheduled_hart 过滤。任务 affinity 可允许任务在多个在线 Hart 上运行，已有远程入队和 IPI/空闲 Hart 通知，但尚无完整主动迁移、work stealing 或负载均衡策略。TID 映射仍为全局锁保护结构。

第四章 内存管理

本章描述当前工作树中 `os/src/mm/`、`os/src/syscall/mm/` 与架构页表代码的已实现行为。Ya2yOS 同时支持 RISC-V64 和 LoongArch64；二者共用地址空间、VMA、用户复制和 COW 的高层接口，页表格式、内核映射和物理 RAM 布局由 `os/src/arch/` 分别实现。

4.1 模块边界与基本对象

位置	当前职责
<code>os/src/mm/address.rs</code>	物理/虚拟地址与页号类型，以及页边界转换。
<code>os/src/mm/frame_alloc/</code>	伙伴式 CMA 物理页分配、FrameTracker 和页缓存接口。
<code>os/src/mm/group.rs</code>	GROUP_SHARE 共享组：mmap 共享 VMA 的 groupid 分配与共享帧登记。
<code>os/src/mm/heap_allocator.rs</code>	静态内核堆与 ContinuousPages。
<code>os/src/mm/map_area.rs</code>	MapArea、映射权限、映射类型与 mmap 文件元数据。
<code>os/src/mm/memory_set/</code>	MemorySet 锁封装、ELF 装载、fork/COW、VMA 操作、mmap 与缺页分发。
<code>os/src/mm/remote_tlb.rs</code>	跨 hart TLB shutdown：全局更新锁、per-hart mailbox、IPI 与确认协议。
<code>os/src/mm/page_fault_handler.rs</code>	匿名/文件 mmap 缺页、COW 写保护缺页和文件 EOF 判断。
<code>os/src/mm/translate.rs</code>	用户地址校验、跨页复制、按需分配触发和安全 VA 到 PA 转换。
<code>os/src/mm/uaccess.rs</code>	per-hart uaccess 状态、Scope、fixup/retry 与同步内核 fault 分类。
<code>os/src/mm/shm.rs</code>	System V 共享内存段的创建、附加、分离和删除。
<code>os/src/mm/mmap_bad_address.rs</code>	mmap 坏地址表：if_bad_address 与坏地址的插入、移除。
<code>os/src/arch/*/qemu/page_table.rs</code>	RISC-V Sv39 与 LoongArch 页表项、激活、COW 与 TLB 操作。

一个 MemorySet 代表一个可切换的虚拟地址空间。它以内部 RwLock 保护 MemorySetInner，后者拥有硬件 PageTable、按创建顺序保存的 Vec<MapArea> 和 total_mmap_size 计数。Process 保存 Arc<MemorySet>；调用者通过 get_ref/get_mut 或 with_ref/with_mut 取得短生命周期的读写保护，不应持有地址空间锁跨越文件系统、调度或信号递送路径。

```
pub struct MemorySet {
    inner: RwLock<MemorySetInner>,
}

pub struct MemorySetInner {
    pub page_table: PageTable,
    pub areas: Vec<MapArea>,
}
```

```
pub total_mmap_size: usize,
}
```

MapArea 是连续虚拟页范围的逻辑描述，记录权限、Direct 或 Framed 映射方式、区域用途、每页的 Arc<FrameTracker>、mmap 后备文件与偏移、mmap flags 和 MAP_SHARED 使用的 group ID。PTE 只保存 VPN 到 PPN 的硬件转换；data_frames 则保存帧的 RAII 所有权和共享引用计数，因而是 COW、共享映射和释放物理页的依据。

MemorySet 锁保护的页表与 VMA 集合	MapArea 范围、权限、后备对象、帧引用	FrameTracker 物理页的零填充与 RAII 回收	PageTable 架构相关的 VPN 到 PPN 转换
---------------------------------	-------------------------------	--------------------------------------	-------------------------------------

图 5 地址空间的高层对象关系。

4.2 架构与地址布局

两种架构均采用 4 KiB 页面，用户空间上限为 0x30_0000_0000。从高地址向下，布局由 trap context 页、每线程用户栈及 guard page、MMAP_TOP 以下的 mmap 区域构成；ELF 主程序从其 program header 指定的地址装载，初始 brk 位于 ELF 映射末端的一页 guard page 之后。USER_STACK_SIZE 为 8 MiB；RISC-V 内核栈为 4 页，LoongArch 内核栈为 2 页。

项目	RISC-V64 QEMU	LoongArch64 QEMU
物理 RAM	0x8000_0000 起连续 2 GiB	总计 2 GiB，低端 0x0000_0000..0x1000_0000 与高端 0x8000_0000..0xf000_0000 两段，中间为 PCI/MMIO hole。
内核堆	48 MiB 静态 HeapSpace	128 MiB 静态 HeapSpace，确保内核镜像留在低端 RAM。
直接映射	KERNEL_ADDR_OFFSET = 0xffff_ffc0_0000_0000	KERNEL_ADDR_OFFSET = 0x9000_0000_0000_0000；内核窗口和 MMIO 由 LoongArch 配置处理。
页表格式	Sv39；内核物理直映射尽可能使用 1 GiB、2 MiB 大页。	LoongArch 页表与 TLB 机制；高层 MapPermission 映射为 LAPTE flags。

RISC-V 启动页表只覆盖第一个 1 GiB。init_cma() 因此先把内核镜像之后、首 GiB 以内的 RAM 加入 CMA；activate_kernel_space() 建立完整内核映射后，init_cma_late() 再加入第二个 GiB。这样伙伴分配器写入自身 free-list 元数据时不会访问启动期尚未映射的物理页。LoongArch 使用 PHYSICAL_MEMORY_RANGES 分段纳管，不会把 PCI/MMIO hole 误作为 RAM。

内核初始化顺序为：初始化静态内核堆，初始化 CMA，激活内核页表，补充 RISC-V 后段 CMA，然后执行 remap_test()。RISC-V 的测试检查内核映射权限；LoongArch 当前实现将该测试作为空操作。

4.3 映射区域、页表与物理帧

MapPermission 统一表示 R/W/X/U。RISC-V 以 RVPTEFlags 编码有效、读写执行、用户、访问、脏和软件 COW 位；LoongArch 以 LAPTEFlags 编码有效、PLV、可写、缓存属性、不可读/不可执行和软件 COW 位。两套实现都将 COW 置于软件可用的 bit 9。

类型	含义	典型来源
Direct	VPN 通过内核页号偏移直接得到 PPN。	RISC-V 内核物理直映射。
Framed	逐页从 CMA/页缓存分配 FrameTracker，并保存到 data_frames。	ELF、用户栈、brk、mmap、共享内存和内核动态区域。
MMIO	按 MMIO_MAP_OFFSET 计算设备物理页，不建立帧追踪器。	架构定义的 UART、块设备或 PCI 相关区域。

FrameTracker::alloc() 从页缓存获得一页并在构造时清零；最后一个 Arc<FrameTracker> 销毁时经页缓存归还。MapArea::map_one() 对 framed 页分配 tracker 并建立 PTE，unmap_one() 删除 PTE 与相应 tracker。push() 立即映射整个区域，push_lazily() 只登记 VMA，push_with_given_frames() 将既有的共享帧映射到新的 VMA。

CMA 是全局的伙伴式连续物理页分配器，页缓存以 32/128 页为低、高水位，空时批量补充 16 页，超过高水位时最多刷回 64 页。RISC-V 启动页表只覆盖首个 1 GiB，所以 init_cma() 先纳管可访问部分，完整内核直映射激活后再由 init_cma_late() 加入其余 RAM；LoongArch 则遍历分段物理范围，避免把 PCI/MMIO hole 当作 RAM。这解决的是启动映射和物理布局差异，不是 NUMA 感知分配。CMA OOM 会使帧分配返回失败，用户缺页随后无法建立映射并由 trap 层转为相应错误信号。

用户页表由 PageTable::new_from_kernel() 创建。RISC-V 路径复制内核高地址部分的根页表项，使陷入内核后可继续访问内核映射；架构相关激活函数在切换地址空间时写入页表根并刷新 TLB。该共享的是内核映射结构，而用户 VMA、用户页表下层和用户 MapArea 仍属于各自 MemorySet。

4.4 多核与异构架构下的一致性

这里的“异构”指同一套内存管理抽象运行在 RISC-V64 与 LoongArch64 两种架构上，而不是已经实现了 NUMA、多种内存一致性域或异构内存节点。MemorySet、MapArea、MapPermission、缺页和 COW 逻辑共用；页表项格式、TLB 指令、权限位和物理 RAM 布局由 os/src/arch/ 的实现分别适配。RISC-V 使用 Sv39 和内核物理直映射，LoongArch 使用自己的页表/TLB 机制，并用 PHYSICAL_MEMORY_RANGES 跳过 PCI/MMIO 空洞。物理页目前仍由全局 CMA 与页缓存管理，不按 hart、架构或 NUMA 节点分区。

多 hart 共享一个 MemorySet 时，active_harts 是该地址空间当前可能仍被硬件访问的 hart 位图。用户返回前，activate_for_user() 在持有地址空间读锁时安装页表并发布当前 hart；调度离开时先清除对应位。页表写者据此决定是否广播失效请求。会替换或移除 PPN 的操作（COW、munmap、mremap、fork/exec 回收等）还会暂时保留旧的 Arc<FrameTracker>，直到所有目标 hart 完成确认，避免远端仍持有旧 TLB 时物理页被重新分配。只改变同一 PPN 的权限或脏位时，旧翻译至多更严格，可以走不保留帧的更新路径。

更新协议由 remote_tlb::UPDATE_LOCK 与 MemorySet 写锁组成，顺序固定为 UPDATE_LOCK -> MemorySet 写锁 -> 子模块锁；禁止在持有 MemorySet guard 时反向获取 UPDATE_LOCK，也不在地址空间锁内进入文件系统、调度、网络或信号路径。新建 lazy PTE 的缺页快路径不会留下旧的有效翻译，因此不必广播；真正替换 PPN 或修改权限则执行 shutdown。发起 hart 先刷新本地 TLB 和指令缓存，再为所有活动远端 hart 准备带 sequence 的 mailbox，并一次性发送 IPI。目标 hart 在 trap/用户返回等可重入点轮询 mailbox，执行本地 TLB invalidate 与 instruction fence 后写入 acknowledged；等

待者会重新读取活动位图，已脱离该地址空间的 hart 不再阻塞协议。可执行页也必须做指令同步，以防 hart 复用旧的译码指令流。

因此，异构支持的核心是“共用地址空间生命周期与一致性协议，架构层提供页表、TLB、IPI 和用户访问原语”，而不是把两种架构强行统一为同一套硬件页表操作。

4.5 ELF、brk 与按需分配

`MemorySetInner::from_elf()` 创建带内核映射的新地址空间，解析 ELF 并映射每个 `PT_LOAD` 段。段权限来自 ELF flags 加用户权限；文件字节被复制到 framed 页面，`mem_size` 超出 `file_size` 的尾部保留为清零内容。若存在 `PT_INTERP`，加载器还会映射动态解释器并把实际入口改为解释器入口；`AT_ENTRY` 仍描述主程序入口，`auxv` 同时提供 `AT_PHDR`、`AT_PHENT`、`AT_PHNUM`、`AT_PAGESZ`、`AT_BASE` 等启动信息。

ELF 末端与 `brk` 之间保留一页 `guard`。`brk` VMA 从零长度开始，`sys_brk()` 通过 `TaskControlBlock::growproc()` 调整范围；增长不立即分配物理帧，首次访问才处理缺页。单进程 `brk` 增长上限为 `MAX_BRK_SIZE = 2 GiB`，架构配置中的用户堆保留范围 `USER_HEAP_SIZE = 2 GiB`。这两个值与独立的 `MAX_MMAP_SIZE = 2 GiB` 预算不同；它们描述虚拟地址空间限制，不代表启动时已经分配对应物理内存。收缩时，`MemorySetInner::grow()` 会解除新末端之后已经存在的 PTE 并释放相应 tracker。

`MemorySetInner::handle_page_fault()` 首先查找覆盖 VPN 的 VMA。未映射页的 `read`、`write` 或 `fetch fault` 在权限允许时走以下路径：`Brk` 与 `Stack` 分配匿名零页；`Mmap` 进入文件/匿名 `mmap` 缺页处理。已存在但写保护的页只在 `store` 或 `page-modify fault` 上尝试 COW 或写权限恢复，读/取指权限错误不会被误作 COW 处理。

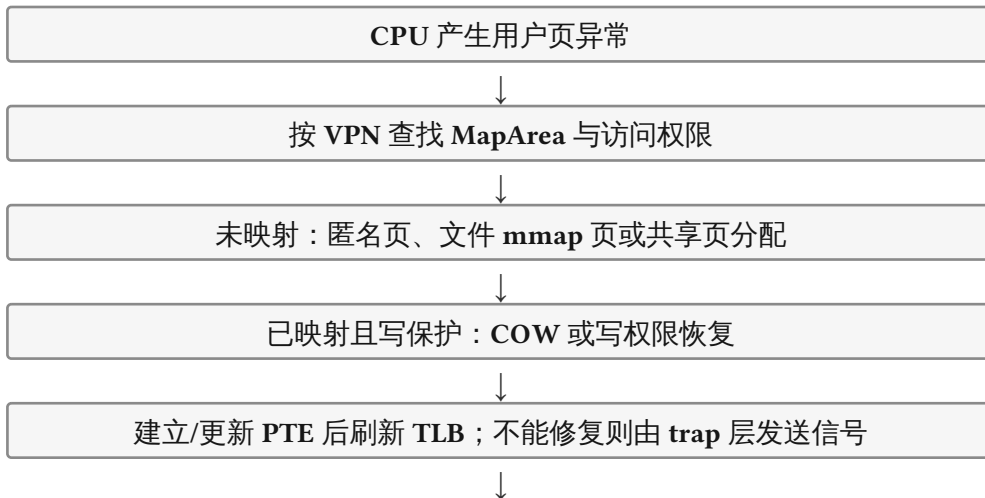


图 4 当前用户缺页处理的高层分支。

文件 `mmap` 的 EOF 语义按 backing inode 的当前长度判断。最后一个部分页面可以零填充；若 `fault` 页的起始文件偏移已在当前 EOF 之外，`mmap_file_page_beyond_eof()` 令 `trap` 层报告 `SIGBUS`，而不是错误地建立零页。建立或替换 VMA 时会先让 `ext4 inode` 记录当前长度，因此 `unlink` 后仍可使用打开 `inode` 的长度；后续 `write/truncate` 更新该长度，文件扩展后新覆盖的页面不会被过期的 VMA 快照误判为 `SIGBUS`。

4.6 fork 与写时复制

非 `CLONE_VM` 的 `clone` 通过 `MemorySetInner::from_existed_user()` 构造子地址空间。它先创建新用户页表，再按 VMA 类型处理父地址空间：`Stack` 与 `Trap` 不直接复制，由任务创建路径为

子线程建立独立资源；Shm 和 MAP_SHARED 复用已有 frames；ELF、brk、私有 mmap 等可写 framed 页面则建立软件 COW 关系。

fork 前，匿名或文件后备的 MAP_SHARED VMA 会被预先 fault：否则父子之后都可能各自为同一延迟页分配不同物理帧，破坏共享可见性。共享 VMA 以 groupid 关联 GROUP_SHARE；首次 fault 把 frame 登记到该组，后续同组 VMA 克隆该 Arc。组内最后一个 MapArea 被释放时，group ID 和共享帧一起释放。

对于 COW 页，父子 PTE 均去除写权限、设置软件 COW，并共享 Arc<FrameTracker>。写 fault 时，若当前 frame 已唯一引用，只需恢复 PTE 写权限；若仍被多个 VMA 持有，则分配新零页、复制旧页内容、把当前 VMA 的 PTE 改指向新页并恢复写权限。此处理同时覆盖 RISC-V store page fault 与 LoongArch 的 page-modify 相关路径。

区域	fork 行为
Stack / Trap	不由 from_existed_user() 复制；子任务建立独立栈与 trap context。
Elf / Brk	复制 VMA 元数据和帧引用；可写页变为 COW。
MAP_PRIVATE	按已有已分配页共享后建立 COW，尚未 fault 的页仍保持延迟状态。
MAP_SHARED	预先 materialize 并通过 GROUP_SHARE/共享 Arc<FrameTracker> 保持父子可见性。
Shm	复用 System V 段保存的帧，不使用 COW。

4.7 mmap、munmap 与 mprotect

sys_mmap() 检查长度、对齐、flags、偏移与文件读写权限后，调用 MemorySet::mmap()。匿名映射必须包含 MAP_ANONYMOUS；文件映射保存 OSFile、文件偏移和映射时的大小快照。普通映射从 MMAP_TOP 向低地址寻找空洞，登记为延迟 MapArea，并按虚拟长度累计到 total_mmap_size。该延迟 VMA 预算受 MAX_MMAP_SIZE = 2 GiB 限制；实际物理页仍只在缺页时分配。MAP_STACK 使用 MapAreaType::Stack；其他 mmap 使用 MapAreaType::Mmap。

MAP_FIXED 与 MAP_FIXED_NOREPLACE 使用调用者指定地址。后者若与既有 VMA 相交，内部返回失败，syscall 映射为 EEXIST；固定映射在需要时调用 VMA 拆分/权限更新路径或创建新的延迟 VMA。当前固定映射不计入 total_mmap_size，这是实现上的可见限制。

MemorySetInner::munmap() 只处理 MapAreaType::Mmap。对于完整覆盖的 VMA，它解除页面映射并删除 VMA；对于部分覆盖，则保留前后片段并转移相应 data_frames。可写的 MAP_SHARED 文件 VMA 在文件仍有链接时，把实际已分配的连续帧范围分块（64 KiB）写回后备文件，随后刷新 TLB。mprotect() 根据边界将 VMA 拆成最多三段，更新目标段的 map_perm 和既有 PTE；它不把未分配页面 materialize。系统调用入口要求地址与长度页对齐，并检查 addr + len 溢出。

madvise() 先要求整个范围被现有 VMA 覆盖。MADV_NORMAL、RANDOM、SEQUENTIAL 和 WILLNEED 完成参数兼容检查；MADV_DONTNEED 对匿名 brk 和私有 mmap 释放驻留帧而保留 VMA，后续访问重新缺页，MAP_SHARED 和固定 ELF 映射按实现例外保留。mlock/munlock/mlockall/munlockall/mlock2 目前只有参数校验路径：由于没有 swap，已分配页本来就驻留，因此不建立真正的锁页记账和资源限制。

mremap() 由 MemorySet::mremap() 根据 flags 分派到原地扩展/收缩或可移动迁移路径。完整覆盖的 Mmap（以及当前实现允许的 Shm）区域可以原地调整；带 MREMAP_MAYMOVE 时选择不相交的新范围，先复制所有已驻留页，成功后才卸载旧映射，未驻留页仍保持 lazy fault。MAP_SHARED 的驻留页也

会深复制到目标帧，因而当前语义不是跨地址范围继续共享同一 PPN；MAP_SHARED_VALIDATE 明确返回 ENOSYS。MREMAP_FIXED 需要同时带 MREMAP_MAYMOVE，目标范围不能与源范围重叠，并会先处理目标区已有映射。所有可能替换/释放 PPN 的路径都通过地址空间更新协议刷新 TLB。mincore() 检查范围覆盖和读权限，并按 PTE 是否存在返回驻留位；内核没有 swap，已映射页即视为驻留。

发起者	关键交互	处理对象
用户态	mmap / munmap / mprotect 请求	syscall 入口
syscall 入口	参数校验、文件权限和用户指针处理	MemorySet
MemorySet	登记、拆分或移除 VMA；按需写回共享文件页	页表 / 文件系统

图 6 VMA 系统调用的当前交互。

4.8 uaccess 设计方案

用户指针不能在 syscall 中直接当作内核指针解引用。当前方案把访问分成“检查、按页转换、架构 fast path”三层。checked_user_range() 先拒绝空首地址、非规范虚拟地址和整数溢出；随后扫描所有覆盖页的 VMA 权限。普通 copy_from_user()/copy_to_user() 按页取得 PPN，缺页时调用 MemorySet::handle_page_fault()，写入时还显式处理 present COW PTE，最终以 EFAULT 表示地址、权限或缺页失败。

translate_user_va_safe() 先用一次安全读触发 lazy allocation，再返回已经存在的 VA 到 PA；只用于已确认映射的内部路径的 direct helper 则不处理缺页、不处理 COW，调用者必须先保证页面有效。这一划分避免把“查询物理地址”误写成会隐式分配的操作。

对当前 hart 正在运行、且长度不超过 256 字节的小型复制，translate.rs 会尝试架构 uaccess fast path。RISC-V 在 Scope 期间临时启用 sstatus.SUM，LoongArch 使用对应的汇编复制原语；两者都由 Scope 记录当前 MemorySet 和 fault fixup 地址，退出时恢复 per-hart 状态。更大的缓冲区、非当前地址空间或 fast path 失败时，回到软件按页转换路径，因而不会让大块 I/O 长时间占用地址空间写者需要的锁。

直接复制引起的同步内核 fault 由 os/src/mm/uaccess.rs 的每 hart 状态接管。状态含 active、地址空间指针、fixup PC 和一次性 retry_vpn：若故障页已经有允许当前访问的 present PTE，只把它视为本地陈旧 TLB，刷新一次后重试；若是缺页、文件后备页、COW 或权限不满足，则跳转 fixup。fixup 不会在 trap frame 中进入可能阻塞的普通缺页解析器，调用方随后在可阻塞的软件路径中重新执行。无法确认是当前任务的地址空间、用户地址范围之外的内核故障或其他异常则返回 Unhandled，保留内核错误处理路径。这样既避免了直接访问用户内存的安全问题，也避免在同步 fault 中重新取得任务锁或阻塞文件系统而形成死锁。

4.9 System V 共享内存与用户复制

shm_create() 在全局 ShmManager 中一次性分配 Vec<Arc<FrameTracker>>，返回 key；shm_attach() 把这些既有帧作为 MapAreaType::Shm 通过 push_with_given_frames() 映射到当前进程，地址为零时从 MMAP_TOP 向下选择地址。shm_detach() 要求地址页对齐并按 VMA 起始页移除映射；shm_drop() 删除全局段记录。当前 MemorySet::shm() 对非零指定附加地址会 panic，因此固定地址 shmat 不是可用的兼容返回路径，文档和测试应将其视为需要修复的危险边界。

所有 syscall 用户指针通过 copy_from_user、copy_to_user 或其 typed wrapper 访问。这些函数拒绝空首地址、不可表示的规范虚拟地址和范围溢出，并按页复制；遇到尚未映射的合法用户页时，会带 Load 或 Store fault 调用 MemorySet::handle_page_fault()。写入路径还会触发 COW 处

理。LoongArch 额外使用 VMA 权限检查；两种架构最终都以失败返回 EFAULT，而不是直接解引用用户虚拟地址。

`translate_user_va_safe()` 在地址转换前先通过读取触发需要的懒分配，适用于 `futex` 等必须取得物理地址的调用点；`translate_va()` 则只查询现有 PTE，不会隐式分配。内部缺页处理、写回等已确认映射存在的路径可使用 `direct read/write helper`，以避免用户复制函数再次取锁。

4.10 当前边界

主题	当前实现边界
物理页回收	无 swap；FrameTracker 最后引用释放后归还 CMA。
多核一致性	<code>active_harts</code> 、全局更新锁和 mailbox/ IPI shutdown 保证共享地址空间的旧 TLB 在帧回收前失效；当前没有 per-hart 或 NUMA 本地分配器。
uaccess 快路径	仅当前 hart 的活动地址空间和不超过 256 字节的小复制使用架构原语；缺页/COW/文件 fault 回退到可阻塞的软件按页路径。
mlock 系列	无 swap 时已分配页天然驻留； <code>mlock/munlock/mlockall/munlockall/mlock2</code> 仅做参数校验并 no-op。
异构架构	RISC-V64 与 LoongArch64 共用 VMA/COW/uaccess 高层协议，但 PTE、权限编码、TLB、IPI 和物理 RAM 分段分别实现；未实现 NUMA 拓扑或不同内存一致性域。
mmap 地址空间	普通 mmap 有 2 GiB 延迟 VMA 计数上限；固定 mmap 不计入该计数。
mremap	支持完整 Mmap/Shm 区域的原地调整和 <code>MREMAP_MAYMOVE</code> 迁移；驻留页先复制后提交， <code>MAP_SHARED_VALIDATE</code> 与 <code>MREMAP_DONTUNMAP</code> 未实现；fixed 必须配合 <code>MAYMOVE</code> 。
SysV shmat	仅自动选址；显式非零地址尚未实现。
文件 mmap EOF	完整页落在映射时 EOF 外会发 SIGBUS；最后一个部分页允许零填充。
跨架构差异	高层 VMA/COW 接口通用，PTE 格式、TLB 和内核物理映射依 RISC-V/LoongArch 不同。

实现追溯：本章对应 `os/src/mm/`、`os/src/syscall/mm/`、`os/src/mm/remote_tlb.rs`、`os/src/mm/uaccess.rs` 以及 `os/src/arch/riscv64/`、`os/src/arch/loongarch64/` 的当前实现；异构与多核部分描述的是共享抽象、分架构适配和现有 hart shutdown 协议，不扩展为尚未实现的 NUMA 能力。

第五章 信号机制

本章对应 `os/src/signal/`、`os/src/syscall/signal.rs`、`os/src/trap/mod.rs` 与任务模块中的等待/状态转换逻辑。信号动作是进程级资源，pending 集、掩码、等待中断标记和备用信号栈是线程级状态；递送在返回用户态前完成。

5.1 模块与数据模型

位置	当前职责
<code>signal/types.rs</code>	Linux 编号、SigSet、用户 ABI SigAction/SigInfo、备用栈和默认动作。
<code>signal/action_table.rs</code>	SigTable：线程组共享的每信号 disposition 与用户 action。
<code>signal/delivery.rs</code>	投递、UID/会话权限检查、线程组和进程组选择、唤醒 stopped/blocked 任务。
<code>signal/pending.rs</code>	选择未屏蔽 pending 信号并执行默认动作或 handler 分发。
<code>signal/frame.rs</code>	用户栈/备用栈信号帧构造及 <code>rt_sigreturn</code> 恢复。
<code>signal/timer.rs</code>	ITIMER_* 过期时的信号投递。
<code>syscall/signal.rs</code>	<code>rt_sigaction</code> 、mask、等待、发送和返回相关 syscall 的 ABI 检查。

SigSet 是一个 `usize` 位图，定义了 1-31 号标准信号、SIGRTMIN 和一个项目使用的实时扩展位。每个 Process 持有 SigTable；表项将用户可见的 SigAction 与内部 SigDisposition:: {Default, Ignore, Handler} 分开保存。默认动作不能用内核函数指针编码，因此用户通过 SIG_DFL 查询或恢复 action 时仍看到 Linux ABI 规定的值。

TaskControlBlockInner 保存 `sig_pending`、`sig_pending_info`、`sig_mask`、`sig_eintr`、SignalStack 和等待中断状态。标准信号不排队：重复投递保留原 pending 位和第一次保存的 SigInfo；因此自定义 SA_SIGINFO handler 可观察到最初发送者的 PID 和 real UID。

SigTable (进程级) action/disposition	TCB (线程级) pending、mask、 SigInfo、alt stack	trap_return 选择信号、默认动作或信号帧	用户 handler rt_sigreturn 恢复上下文
--------------------------------------	---	---------------------------	----------------------------------

图 7 信号状态的归属与递送边界。

5.2 发送、权限与唤醒

`kill`、`tkill` 和 `tgkill` 均校验信号编号；`kill(pid, 0)` 只进行存在性和权限探测。`kill` 支持指定线程组、当前/指定进程组和所有可访问进程的选择语义，`tkill` 指向线程，`tgkill` 同时验证目标 TID 隶属于指定 TGID。用户态发送路径比较发送者的 real/effective UID 与目标 real/saved UID；effective UID 为 0 可越过普通比较，同一 session 的 SIGCONT 也允许发送。成功的用户态投递生成带发送者 PID/UID 的 SigInfo。

内核内部路径（页故障、子进程状态变化、定时器）使用不带用户发送者信息的 helper。`add_signal_with_info()` 将信号置为 pending 后按状态处理：SIGCONT 或 SIGKILL 可将 Stopped 任务恢复到 Ready；可中断阻塞任务优先经 `interrupt_waker` 唤醒，否则重新加入 ready queue。对

默认可忽略的 SIGCHLD，只有安装 handler 时才作为可中断 等待处理，避免无意义地使阻塞 syscall 返回 EINTR。

退出和等待的通知状态存于 ProcessMeta，而非 SigTable。最后一个线程退出时会向父 线程组发送 SIGCHLD 并唤醒 child_exit_event；stop/continue 事件同样可唤醒 waitpid/waitid 的重新扫描。

5.3 递送与默认动作

trap_return() 在恢复用户寄存器前循环检查 sig_pending & !sig_mask，每轮选择编号 最小的信号。若本轮建立用户 handler 帧则立即返回用户态，让 handler 先运行，避免连续递送覆盖尚未恢复的信号帧。SIGKILL 与 SIGSTOP 始终从掩码移除，rt_sigaction 也拒绝修改二者的 disposition。

disposition	递送结果
Handler	在当前用户栈或启用的备用栈构造帧，跳转到 sa_handler。
Ignore	消费 pending 位，不改变用户执行流。
Default: Terminate/ CoreDump	以 128 + signo 调用任务退出路径；当前不生成 core 文件。
Default: Stop	任务进入 Stopped，等待 SIGCONT 或 SIGKILL。
Default: Continue	恢复停止任务并记录可供父进程等待的 continue 事件。

来自用户页故障的不可恢复访问通常转为 SIGSEGV；文件 mmap 的映射时 EOF 之外页面 转为 SIGBUS。非法指令当前直接走任务退出路径，未支持的 trap 仍会 panic，不能表述为 所有硬件异常都已具备完整 POSIX 信号语义。

5.4 用户信号帧与返回

当 action 为 handler 时，setup_frame() 先选择可用的备用栈 (SA_ONSTACK 且栈已 启用) 或当前用户栈，检查空间与用户地址有效性，再保存 machine context、旧掩码和 SigInfo。随后它根据 SA_SIGINFO 设置 handler 参数，按照 SA_NODEFER 更新当前 掩码，并将返回地址设置为 libc restorer 或架构提供的 sigreturn_trampoline。SA_RESETHAND 在进入 handler 时将该 action 复位为默认；SA_RESTART 的可重启判断 由帧恢复路径结合原 syscall context 处理。

sys_rt_sigreturn() 调用 restore_frame()，从受检用户内存读取信号帧，恢复保存的 trap context、信号掩码和备用栈状态，并返回原始 a0。帧解析失败返回 EFAULT，避免 直接信任用户提供的栈内容。最新 trampoline 调整后，restorer 地址、用户态返回入口和 架构 trap context 必须作为一个 ABI 整体核对：RISC-V 和 LoongArch 的低层布局不同，但高层 action、pending 与恢复协议共用。

发起者	关键交互	处理对象
内核/用户发送者	置 pending、记录 SigInfo、唤醒目标	目标 TCB
trap_return	选择未屏蔽信号并构造 frame	用户 handler
用户 handler	rt_sigreturn	恢复 trap context 与 mask

图 8 信号从投递到用户态恢复的当前路径。

5.5 相关系统调用与限制

`rt_sigaction` 使用架构相关的 `RawSigAction` 布局与用户态交换 `action` ; `rt_sigprocmask`、`rt_sigpending`、`rt_sigsuspend` 和 `rt_sigtimedwait` 操作线程级 `pending/mask`。 `rt_sigtimedwait` 在用户指定集合中消费最小编号 `pending` 信号，并通过任务定时器返回 `EAGAIN` ; `rt_sigsuspend` 在收到未屏蔽信号后恢复旧 `mask` 并返回 `EINTR`。 `sigaltstack`、`setitimer`/`getitimer` 和 POSIX timer 的实现分别位于 `signal` 和 `timer` 路径。

当前限制包括：标准信号不维护 Linux realtime queue ; `core-dump` 默认动作只终止；`signalfd4` 仍是兼容性 `fd`，不能替代完整的 `signal-fd` 消费语义；未支持的硬件 trap 可能 panic。修改该路径时必须避免持有 `task/process` 锁跨越调度和用户内存访问，并验证阻塞 `syscall` 的 `EINTR`、`SA_RESTART` 与 `stop/continue` 交互。

实现追溯：`os/src/signal/`、`os/src/syscall/signal.rs`、`os/src/trap/mod.rs`

第六章 网络模块

6.1 概述

Ya2yOS 的网络模块位于 `os/src/net/` 与 `os/src/syscall/net/`，整体沿用了 ArceOS 风格的 `smoltcp` 集成方式，并结合当前内核的文件描述符、任务阻塞和 `VirtIO` 设备层做了适配。网络功能受 `feature = "net"` 控制；启用后，`socket fd` 通过 `FileClass::Socket` 纳入统一的文件描述符模型。

当前网络模块的目标是提供 Linux `socket API` 的主要兼容面，而不是完整复刻 Linux 网络栈。已实现的重点包括 IPv4 `TCP/UDP`、`loopback`、`VirtIO-net`、`Unix domain socket`、常见 `socket syscall`、`poll/epoll` 接入和部分 `socket option`。

6.2 架构分层

网络模块可以分为四层：

syscall/net
socket/bind/listen/accept/connect/send/recv getsockopt/setsockopt/getpeername/...
net/socket.rs + tcp.rs + udp.rs + unix.rs SocketOps、Socket 枚举、TCP/UDP/Unix 实现
service.rs + router.rs + wrapper.rs smoltcp Interface、SocketSet、路由与调度
net/device + drivers/virtio/net.rs LoopbackDevice、EthernetDevice、VirtIO-net

`Service` 持有 `smoltcp Interface` 和自定义 `Router`；`Router` 同时实现 `smoltcp phy::Device`，将协议栈的 IP 包分发到 `loopback` 或以太网设备。

6.3 全局对象

6.3.1 SOCKET_SET

```
static SOCKET_SET: Lazy<SocketSetWrapper> =
    Lazy::new(SocketSetWrapper::new);
```

`SOCKET_SET` 封装 `smoltcp SocketSet`，用于保存 `TCP/UDP` 的底层 `smoltcp socket`。`TCP/UDP` 对象只保存 `SocketHandle`，真正的数据缓冲区和协议状态在 `SocketSet` 中。

6.3.2 LISTEN_TABLE

```
static LISTEN_TABLE: Lazy<ListenTable> = Lazy::new(ListenTable::new);
```

`LISTEN_TABLE` 记录监听中的 `TCP` 端口。`Router::snoop_tcp_packets()` 会在 IP 包进入 `smoltcp` 前识别 `TCP SYN` 包，并通知监听表为 `accept()` 准备已连接 `socket handle`。

6.3.3 SERVICE

```
static SERVICE: Once<Mutex<Service>> = Once::new();
```

SERVICE 是网络模块的调度核心。poll_interfaces() 会反复调用 Service::poll(), 驱动收包、TCP 状态机、重传定时器和发包队列。

6.4 初始化流程

init_network(net_devs) 在内核启动时初始化网络：

1. 创建 Router；
2. 注册 LoopbackDevice，配置 127.0.0.1/8；
3. 如果探测到 VirtIO-net 设备，则创建 EthernetDevice("eth0")，使用 net/consts.rs 中的 IP、前缀长度和网关；
4. 添加两类路由规则：127.0.0.0/8 走 loopback，默认路由 0.0.0.0/0 走 eth0 和网关；
5. 创建 Service，将 loopback IP 和 eth0 IP 写入 smoltcp Interface；
6. 将 Service 保存到全局 SERVICE。

RISC-V64 上 VirtIO-net 通过 MMIO 地址探测；LoongArch64 上通过 PCI 枚举得到 PciTransport。如果没有物理网卡，网络模块仍可保留 loopback 能力。

6.5 路由与设备

6.5.1 Router

Router 内部维护接收/发送包缓冲区、设备列表和路由表：

```
pub struct Router {
    rx_buffer: PacketBuffer,
    tx_buffer: PacketBuffer,
    devices: Vec<Box<dyn Device>>,
    table: RouteTable,
}
```

路由表按前缀长度排序，查找时使用最长前缀匹配。dispatch() 从 tx_buffer 取出 smoltcp 生成的 IP 包，对广播/组播/单播分别处理，并根据路由规则选择下一跳和出口设备。

6.5.2 LoopbackDevice

LoopbackDevice 在内存中模拟 lo：发送时将包写入内部 FIFO，接收时从 FIFO 取出并交给协议栈。它用于本机进程间的 TCP/UDP 回环通信。

6.5.3 EthernetDevice

EthernetDevice 包装 NetDeviceImpl，提供以太网帧收发、ARP 缓存和组播 MAC 映射：

- IPv4 包会被封装为 Ethernet frame 后发送；
- 目标 MAC 未知时发送 ARP 请求，并暂存待发送 IP 包；
- 收到 ARP 请求时会按本机 IP 返回 ARP reply；
- ARP 缓存默认 TTL 为 60 秒；
- IPv4 组播地址会映射到 01:00:5e:* 以太网组播 MAC。

6.6 TCP 套接字

TCP 使用 smoltcp TCP socket 作为底层状态机：

```
pub struct TcpSocket {
    state: StateLock,
```

```

    handle: SocketHandle,
    general: GeneralOptions,
    rx_closed: AtomicBool,
    poll_rx_closed: PollSet,
    memberships: RwLock<Vec<(u32, IpAddress)>>,
}

```

State 包括 Idle、Connecting、Connected、Listening、Closed 和 Busy。TCP RX/TX 缓冲区大小由 net/consts.rs 中的 TCP_RX_BUF_LEN、TCP_TX_BUF_LEN 决定。

服务端流程：

1. socket(AF_INET, SOCK_STREAM, proto) 创建 TcpSocket；
2. bind() 设置本地 IP/端口，端口为 0 时分配临时端口；
3. listen() 将端口加入 LISTEN_TABLE；
4. accept() 通过 LISTEN_TABLE.accept() 获取已完成连接；
5. read/recv 和 write/send 映射到 TCP 接收/发送缓冲区。

客户端流程：

1. connect() 自动选择源 IP 和临时端口；
2. 调用 smoltcp connect() 发起握手；
3. 通过 poll_interfaces() 推进握手，连接成功后状态变为 Connected；
4. 后续收发与普通流式 socket 一致。

TcpSocket 实现了 File，因此 read/write/poll/epoll 可以直接作用在 TCP fd 上。poll() 会根据连接状态返回可读、可写或 RDHUP 事件。

6.7 UDP 套接字

UDP 套接字维护本地端点、可选 peer 和通用选项：

```

pub struct UdpSocket {
    handle: SocketHandle,
    local_addr: RwLock<Option<IpEndpoint>>,
    peer_addr: RwLock<Option<(IpEndpoint, IpAddress)>>,
    general: GeneralOptions,
    memberships: RwLock<Vec<(u32, IpAddress)>>,
}

```

bind() 会打开 smoltcp UDP socket；端口为 0 时分配临时端口。connect() 只记录默认远端和源地址，不进行握手。sendto/sendmsg 可以指定目标地址，send/recv 使用已连接的 peer。接收路径支持返回源地址和 MSG_TRUNC 语义。

UDP 支持 IP_TTL、组播加入/离开等部分 IP 层选项。

6.8 Unix Domain Socket

AF_UNIX 当前支持 SOCK_STREAM 和 SOCK_DGRAM，包括 socketpair()。Unix socket 不经过 smoltcp，使用内核内存队列传递消息：

```

pub enum UnixSocketAddr {
    Unnamed,
    Abstract(Vec<u8>),
    Path(String),
}

pub enum UnixSocketKind {
    Stream,
}

```

```
Dgram,
}
```

UNIX_BINDS 是全局地址表，负责路径/abstract 地址到 socket 的绑定关系。stream socket 的 listen/connect/accept 通过 pending 队列建立连接；dgram socket 可按目标地址投递消息。每个 socket 的接收队列上限语义仍较简化，register() 尚未接入异步 waker，因此 Unix socket 的 epoll 唤醒能力弱于 TCP/UDP。

6.9 Socket Option

网络模块通过 Configurable trait 分发 socket option：

```
pub trait Configurable {
    fn get_option_inner(&self, opt: &mut GetSocketOption) ->
    SysResult<bool>;
    fn set_option_inner(&self, opt: SetSocketOption) -> SysResult<bool>;
}
```

当前覆盖的主要选项包括：

- SOL_SOCKET：SO_REUSEADDR、SO_SNDBUF、SO_RCVBUF、SO_KEEPALIVE、SO_RCVTIMEO、SO_SNDTIMEO；
- IPPROTO_IP：IP_TTL、MCAST_JOIN_GROUP、MCAST_LEAVE_GROUP，IP_MULTICAST_IF 目前兼容返回成功；
- IPPROTO_TCP：TCP_NODELAY；
- AF_UNIX：发送/接收缓冲区大小、非阻塞标志、peer credentials、pass credentials 兼容。

未支持的选项返回 ENOPROTOOPT。部分 getsockopt 路径仍偏兼容测试导向，并非完整 Linux 语义。

6.10 系统调用与 File 统一

sys_socket() 按 domain/type/protocol 创建 socket：

- AF_INET + SOCK_STREAM -> TCP；
- AF_INET + SOCK_DGRAM -> UDP；
- AF_UNIX + SOCK_STREAM/SOCK_DGRAM -> Unix socket；
- 其他 family/type 返回 EAFNOSUPPORT、EPROTONOSUPPORT 或 EPROTONOSUPPORT。

创建出的 socket 以 FileClass::Socket(Arc<Socket>) 放入 fd 表：

```
pub enum Socket {
    Udp(UdpSocket),
    Tcp(TcpSocket),
    Unix(UnixSocket),
}
```

bind/listen/accept/connect/shutdown 通过 SocketOps 分发；sendto/sendmsg/recvfrom/recvmsg 会解析用户态 msghdr/iovec/sockaddr，复制数据并调用对应 socket 的 send/recv。

6.11 数据收发路径

以 TCP 发送为例：

1. 用户态调用 sendmsg()；
2. syscall 层读取 msghdr 和 iovec，将用户缓冲区整理为 UserBuffer；
3. fd 表查到 FileClass::Socket，分发到 TcpSocket::send()；
4. TcpSocket::send() 调用 smoltcp TCP socket 写入发送缓冲区；
5. poll_interfaces() 驱动 smoltcp 生成 IP 包，并写入 Router.tx_buffer；
6. Router::dispatch() 根据路由选择 loopback 或 eth0；

7. EthernetDevice 执行 ARP/以太网封装，最终调用 VirtIO-net 驱动发送。

接收路径反向进行：设备收到帧后交给 EthernetDevice，IPv4 payload 进入 Router.rx_buffer，smoltcp 消费 IP 包并填充对应 socket 的接收缓冲区，用户态通过 recvmsg() 取出数据。

6.12 当前边界

1. 协议范围：主要覆盖 IPv4 TCP/UDP 和 Unix socket；IPv6 地址解析存在，但完整 IPv6 路由、邻居发现和上层语义并不完整。
2. 驱动方式：网络通过 poll_interfaces() 周期性轮询推进；VirtIO-net 内部已有部分中断确认和 waker 逻辑，但架构 IRQ 抽象仍有未完成的 enable/disable/acknowledge 钩子，尚未形成完整的中断驱动收包与 socket 唤醒路径。
3. **socket option**：只覆盖常见选项，部分选项仅兼容返回，getsockopt 行为还需继续贴近 Linux。
4. **Unix socket** 唤醒：Unix socket 可读写和 socketpair 已有基础实现，但 waker 注册仍是空实现。
5. 高级能力：缺少 netlink、raw socket、packet socket、完整防火墙/路由管理、TCP_INFO 等能力。

第七章 I/O 设备

7.1 设备驱动架构

Ya2yOS 的设备层位于 `os/src/drivers/` ,主要服务块设备和网络设备。驱动接口分为基础设备 trait 与具体设备能力 trait :

```
pub enum DeviceType {
    Block,
    Char,
    Net,
    Display,
}

pub trait BaseDriver: Send + Sync {
    fn device_name(&self) -> &str;
    fn device_type(&self) -> DeviceType;
    fn irq_num(&self) -> Option<usize> { None }
}

pub trait BlockDriver: BaseDriver {
    fn num_blocks(&self) -> usize;
    fn block_size(&self) -> usize;
    fn read_block(&mut self, block_id: usize, buf: &mut [u8]) ->
    DevResult;
    fn write_block(&mut self, block_id: usize, buf: &[u8]) -> DevResult;
    fn flush(&mut self) -> DevResult;
}
```

网络设备还定义了 `NetDriverOps` ,提供 MAC 地址、收发队列状态、收包、发包、TX buffer 分配与回收等接口。`DeviceContainer<D>` 用一个小型容器保存探测到的设备 ,网络初始化时从中取出一个设备作为 `eth0`。

7.2 VirtIO 支持

7.2.1 传输方式

Ya2yOS 使用 `virtio-drivers` crate 驱动虚拟 IO 设备 :

- RISC-V64 使用 MMIO transport。块设备默认位于 `0x10001000 + KERNEL_ADDR_OFFSET` ,网络设备默认位于 `0x10002000 + KERNEL_ADDR_OFFSET` ;
- LoongArch64 使用 PCI transport。内核枚举 PCI 配置空间 ,识别 VirtIO 设备并构造 `PciTransport`。

当前实际使用的 VirtIO 设备包括 `virtio-blk` 和 `virtio-net`。

7.2.2 HAL 与 DMA

`VirtIoHalCMAImpl` 是提供给 `virtio-drivers` 的 HAL。它负责 :

- 通过 CMA 分配物理连续页作为 DMA 缓冲区 ;
- 将物理地址转换为内核虚拟地址 ;
- 在 `share/unshare` 中为设备可见缓冲区分配、拷贝和释放内存 ;
- 在 LoongArch64 上修正 PCI BAR/MMIO 物理地址到 QEMU MMIO 窗口。

这套 HAL 让 virtqueue 描述符可以指向设备可访问的物理内存。

7.2.3 virtqueue 机制

VirtIO 设备使用 virtqueue 与驱动交换请求。典型队列包含 descriptor table、available ring 和 used ring。驱动准备描述符、写入 available ring 并通知设备；设备处理后写入 used ring，驱动再回收请求。

当前块设备读写直接调用 virtio-drivers 的同步接口；网络设备也通过 virtio-drivers 提供的 net raw 接口管理 RX/TX buffer。

7.3 块设备

7.3.1 VirtIO 块设备

不同架构使用不同的块设备封装：

```
#[cfg(target_arch = "riscv64")]
pub type BlockDeviceImpl = VirtIoBlkDev<VirtIoHalCMAImpl>;

#[cfg(target_arch = "loongarch64")]
pub type BlockDeviceImpl = VirtIoBlkDev2<VirtIoHalCMAImpl>;
```

RISC-V64 的 VirtIoBlkDev 包装 VirtIOBlk<H, MmioTransport>；LoongArch64 的 VirtIoBlkDev2 通过 PCI 枚举得到 transport。二者都实现 BlockDriver，块大小固定为 512 字节。flush() 当前是兼容性空操作。

7.3.2 Disk 光标抽象

Disk 在块设备之上提供按字节位置读写的光标：

```
pub struct Disk {
    block_id: usize,
    offset: usize,
    dev: BlockDeviceImpl,
}
```

read_one() 和 write_one() 每次最多处理一个块内的连续片段。若当前 offset 为 0 且缓冲区至少一个块，则直接整块读写；否则先读取整块到临时缓冲，再进行局部复制和写回。set_position(pos) 将字节偏移转换为 block_id + offset。

7.3.3 文件系统接入

ext4 的 lwext4 适配层将 Disk 作为块设备后端，通过 seek/read/write/size 接口让 lwext4 在 VirtIO 磁盘镜像上读写文件系统。文件系统层不直接操作 virtqueue，而是通过 Disk 和 BlockDriver 间接访问块设备。

7.4 网络设备

7.4.1 NetDriverOps

网络设备的底层驱动接口如下：

```
pub trait NetDriverOps: BaseDriver {
    fn mac_address(&self) -> EthernetAddress;
    fn can_transmit(&self) -> bool;
    fn can_receive(&self) -> bool;
    fn rx_queue_size(&self) -> usize;
```

```

fn tx_queue_size(&self) -> usize;
fn recycle_rx_buffer(&mut self, rx_buf: NetBufPtr) -> DevResult;
fn recycle_tx_buffers(&mut self) -> DevResult;
fn transmit(&mut self, tx_buf: NetBufPtr) -> DevResult;
fn receive(&mut self) -> DevResult<NetBufPtr>;
fn alloc_tx_buffer(&mut self, size: usize) -> DevResult<NetBufPtr>;
}

```

网络层的 EthernetDevice 只依赖该 trait，因此可以在 VirtIO-net 之外继续扩展其他 NIC 后端。

7.4.2 VirtIO-net

```

#[cfg(target_arch = "riscv64")]
pub type NetDeviceImpl = VirtIoNetDev<VirtIoHalCMAImpl, MmioTransport,
QUEUE_SIZE>;

#[cfg(target_arch = "loongarch64")]
pub type NetDeviceImpl = VirtIoNetDev<VirtIoHalCMAImpl, PciTransport,
QUEUE_SIZE>;

```

VirtIoNetDev 包装 VirtIONetRaw，队列大小为 QUEUE_SIZE = 128。驱动维护一组 TX buffer 池，发送时分配 buffer、填充以太网帧并调用 transmit()；接收时从设备取出 RX buffer，交给上层处理后调用 recycle_rx_buffer() 归还。

receive() 中会调用 ack_interrupt()，但当前网络栈主要由 poll_interfaces() 周期性轮询推进，而不是完整依赖外部中断。

7.5 控制台与字符类设备

内核控制台由 console.rs 和架构相关串口实现提供。Stdin、Stdout 是文件系统中的抽象文件，进程创建时默认占据 fd 0、1、2：

- Stdin::read() 从控制台读取输入；
- Stdout::write() 将用户输出写到控制台；
- fd 2 当前同样指向 Stdout。

/dev 下的字符设备由文件系统的 devfs 兼容层实现，而不是在 drivers/ 下建立独立字符设备驱动框架。

7.6 LoongArch64 PCI

LoongArch64 平台通过 os/src/drivers/virtio/loongarch/pci.rs 枚举 PCI 设备：

1. 遍历 bus/device/function；
2. 读取 vendor/device ID；
3. 查找 VirtIO 块设备和 VirtIO 网络设备；
4. 配置 BAR 空间；
5. 创建 PciTransport 并交给对应 VirtIO 驱动。

网络设备的 PCI 初始化会设置命令寄存器，配置 32 位或 64 位 BAR，并通过 PciTransport::new 创建 transport。

7.7 中断与轮询策略

当前 IO 路径偏同步和轮询：

- virtio-blk 读写使用同步调用，系统调用或文件系统操作会等待设备请求完成；

- virtio-net 的收发由 `poll_interfaces()` 推进，设备中断只做了部分 `ack/waker` 能力；架构 IRQ 抽象中的硬件 `enable/disable/complete` 路径仍有 `unimplemented!()`，尚未形成完整 NAPI 风格路径；
- 控制台输入输出采用轮询式访问。

这种设计便于在竞赛内核中保持实现简单和可调试，但在高吞吐或低延迟 IO 场景下会产生额外 CPU 开销。

7.8 /dev 兼容层

设备文件由 `os/src/fs/files/devfs.rs` 注册和打开。当前常见路径包括：

- `/dev/null`：读返回 EOF，写丢弃；
- `/dev/zero`：读返回零，写丢弃；
- `/dev/random`：提供兼容性随机数据；
- `/dev/rtc`、`/dev/rtc0`、`/dev/misc/rtc`：简化 RTC；
- `/dev/tty`：终端；
- `/dev/cpu_dma_latency`：记录 CPU 延迟配置；
- `/dev/loop-control`、`/dev/loopN`、`/dev/block/loopN`：loop 设备兼容接口。

这些路径由 `open()` 在进入 `ext4` 普通文件查找前识别，返回 `FileClass::Abs`。

7.9 Loop 设备

Loop 设备当前是面向 Linux 工具和 LTP 的兼容实现。内部维护 256 个 `LoopState`：

```
struct LoopState {
    backing_fd: Option<usize>,
    info: loop_info64,
}
```

支持的主要 ioctl：

- `/dev/loop-control`：`LOOP_CTL_GET_FREE`、`LOOP_CTL_ADD`、`LOOP_CTL_REMOVE`；
- `/dev/loopN`：`LOOP_SET_FD`、`LOOP_CLR_FD`、`LOOP_GET_STATUS*`、`LOOP_SET_STATUS*`、`BLKGETSIZE64`。

当前实现只记录 `backing fd` 和 `loop info`，块设备的 `read()` 返回 0，`write()` 丢弃数据。因此它不能真正把普通文件映射为可读写块设备，也不能作为完整镜像挂载后端。

7.10 当前边界与后续方向

1. 中断驱动 IO：完善 VirtIO 外部中断、`waker` 和网络收包路径，减少轮询开销。
2. 页缓存与块缓存：在文件系统和块设备之间加入统一缓存，减少重复磁盘访问。
3. 真实 loop 数据路径：将 loop 设备读写转发到 `backing file`，支持镜像类测试和真实挂载场景。
4. 更多 VirtIO 设备：继续接入 `virtio-rng`、`virtio-input`、`virtio-gpu` 等设备。
5. 设备模型：建立更统一的设备发现、注册、权限和 `/dev` 节点生成机制。
6. 异步 IO：结合 `io_uring/AIO` 和设备 `waker`，实现更完整的非阻塞 IO 能力。

第八章 文件系统

8.1 概述

Ya2yOS 的文件系统以 VFS 为统一抽象层，将系统调用、普通文件、设备文件、管道、套接字、epoll/eventfd/inotify、信号与消息队列等对象统一纳入文件描述符模型。底层磁盘文件系统采用 lwext4_rust 绑定的 lwext4，在 VirtIO 块设备之上提供 ext4 读写能力。

当前文件系统的主要组成如下：

- vfs.rs 定义 SuperBlock、Inode、File 三类核心 trait；
- ext4_lw/ 将 lwext4 封装为 Ext4Inode 与全局 superblock，并为 C 侧资源锁安装任务感知的睡眠等待；
- dcache.rs 提供 VFS 层目录项缓存，page_cache.rs 提供普通文件读、mmap 与 splice 共享的页级缓存；
- kernel_fs_ops/ 实现 open、FsIndex inode 缓存、初始目录/文件创建、proc 文件刷新和动态链接路径映射；
- files/ 存放各种 File 实现，包括普通文件、管道/FIFO、设备文件、loop 设备、epoll、eventfd、inotify、fanotify、signalfd、mqueue、io_uring 占位、挂载上下文 fd 等；
- fstruct.rs 管理进程级文件描述符表；
- fs_info.rs 维护进程的当前目录、可执行文件路径、fd 到路径映射和 umask；
- mount.rs 维护路径化的挂载层、传播关系和 event group，并同步 /proc/mounts。

8.2 VFS 抽象

8.2.1 SuperBlock

SuperBlock 表示一个文件系统实例的入口。当前实现主要由 ext4 superblock 提供：

```
pub trait SuperBlock: Send + Sync {
    fn root_inode(&self) -> Arc<dyn Inode>;
    fn sync(&self);
    fn fs_stat(&self) -> Statfs;
    fn ls(&self);
}
```

系统通过 superblock_root_inode() 获取根 inode，通过 superblock_fs_stat() 服务 statfs/fstatfs，通过 superblock_sync() 将缓存写回磁盘。

8.2.2 Inode

Inode 表示目录树中的文件系统节点。它负责路径查找、目录项创建、普通读写、元数据、链接与扩展属性操作：

```
pub trait Inode: Send + Sync {
    fn size(&self) -> usize;
    fn types(&self) -> InodeType;
    fn fstat(&self) -> Kstat;
    fn create(&self, path: &str, ty: InodeType) -> Result<Arc<dyn Inode>, SysErrNo>;
    fn create_with_metadata(&self, path: &str, ty: InodeType, mode: u32,
                           owner: Option<(u32, u32)>)
        -> Result<Arc<dyn Inode>, SysErrNo>;
    fn find(&self, path: &str, flags: OpenFlags, loop_times: usize)
```

```

        -> Result<Arc<dyn Inode>, SysErrNo>;
    fn find_from_cached_parent(&self, path: &str, flags: OpenFlags)
        -> Result<Arc<dyn Inode>, SysErrNo>;
    fn read_at(&self, off: usize, buf: &mut [u8]) -> SyscallRet;
    fn write_at(&self, off: usize, buf: &[u8]) -> SyscallRet;
    fn read_dentry(&self, off: usize, len: usize) -> Result<(Vec<u8>,
    isize), SysErrNo>;
    fn is_dir_empty(&self) -> Result<bool, SysErrNo>;
    fn truncate(&self, size: usize) -> SyscallRet;
    fn set_timestamps(&self, atime: Option<u64>, mtime: Option<u64>,
        ctime: Option<u64>) -> SyscallRet;
    fn set_xattr(&self, name: &[u8], value: &[u8], flags: u32) ->
    SyscallRet;
    fn get_xattr(&self, name: &[u8], value: &mut [u8]) -> SyscallRet;
    fn link_cnt(&self) -> SyscallRet;
    fn unlink(&self, path: &str) -> SyscallRet;
    fn read_link(&self, buf: &mut [u8], bufsize: usize) -> SyscallRet;
    fn sym_link(&self, target: &str, path: &str) -> SyscallRet;
    fn rename(&self, path: &str, new_path: &str) -> SyscallRet;
    fn hard_link(&self, old_path: &str, new_path: &str) -> SyscallRet;
    fn read_all(&self) -> Result<Vec<u8>, SysErrNo>;
    fn path(&self) -> String;
    fn page_cache_path(&self) -> Option<Arc<str>>;
    fn cache_identity(&self) -> Option<(usize, usize)>;
    fn fmode(&self) -> Result<u32, SysErrNo>;
    fn fmode_set(&self, mode: u32) -> SyscallRet;
}

```

与早期版本相比，trait 新增了几类接口：

- create_with_metadata() 把 create/mode/owner 合并到一个临界区内，避免分开更新时被并发 open 观察到中间状态；
- find_from_cached_parent() 让已缓存父目录只做末级目录项查找，跳过高成本的全路径遍历；
- is_dir_empty() 供 rmdir/unlinkat(AT_REMOVEDIR) 在删除前保持 Linux 语义，防止后端递归删除；
- set_xattr/get_xattr/list_xattr/remove_xattr 默认返回 EOPNOTSUPP，由 ext4 覆写；
- mark_directory_stat_changed()、touch_atime() 支撑 stat 缓存的按目录失效和 atime 更新；
- page_cache_path() 返回共享的 Arc<str> 路径，供文件页缓存零拷贝构造键；
- cache_identity() 提供 (st_dev, st_ino) 身份键避免额外 fstat()。

目前真正落到磁盘的 inode 实现是 Ext4Inode。部分内核生成文件会复用 ext4 普通文件承载内容，而设备、管道、事件对象等不经过 Inode，直接实现 File。

8.2.3 File

File 是系统调用层面对 fd 操作的统一接口。普通文件、pipe、socket、epoll、eventfd、inotify、fanotify、signalfd、mqueue、设备文件都实现该 trait：

```

pub trait File: Send + Sync {
    fn update_signal_mask(&self, mask: SigSet) -> bool;
    fn readable(&self) -> bool;
    fn writable(&self) -> bool;
}

```

```

fn read(&self, buf: UserBuffer) -> SyscallRet;
fn write(&self, buf: UserBuffer) -> SyscallRet;
fn truncate(&self, size: usize) -> SyscallRet;
fn write_kernel_bytes(&self, buf: &[u8]) -> SyscallRet;
fn fstat(&self) -> Kstat;
fn path(&self) -> Cow<'_, str>;
fn lseek(&self, offset: isize, whence: usize) -> SyscallRet;
fn nonblocking(&self) -> bool;
fn set_nonblocking(&self, nonblocking: bool) -> SysResult;
fn poll(&self, events: PollEvents) -> PollEvents;
fn ioctl(&self, cmd: u32, arg: usize, memory_set: &MemorySet) ->
SyscallRet;
fn register(&self, context: &mut Context<'_,>, events: PollEvents);
}

```

`poll()` 和 `register()` 让普通 `fd` 可以接入 `poll/ppoll/select/epoll` 的事件等待模型；`ioctl()` 默认返回 `ENOTTY`，由设备文件或特殊对象按需覆写。`update_signal_mask()` 用于 `signalfd4` 复用既有 `fd` 时更新信号掩码；`write_kernel_bytes()` 供内核内部向抽象文件注入数据。

8.3 文件描述符与进程文件上下文

8.3.1 FdTable

每个进程拥有一个 `FdTable`，内部通过读写锁保护：

```

pub struct FdTable {
    inner: RwLock<FdTableInner>,
}

pub struct FdTableInner {
    soft_limit: usize,
    hard_limit: usize,
    files: Vec<Option<FileDescriptor>>,
    reserved: Vec<bool>,
}

#[derive(Clone)]
pub struct FileDescriptor {
    flags: OpenFlags,
    file: FileClass,
}

```

默认 `fd` 表带有 `stdin/stdout/stderr` 三项，软上限为 128，硬上限为 256。`alloc_fd()` 使用最小可用编号策略，并在返回前把目标槽位标记为 `reserved`，避免并发线程同时分配到同一 `fd`；随后安装描述符时再清除预留。`dup`、`dup3`、`fcntl(F_DUPFD*)` 复制 `FileDescriptor`，共享底层 `Arc<File>`；`close_on_exec()` 会关闭带 `O_CLOEXEC` 的描述符。

`FileClass` 区分不同 `fd` 类型：

```

pub enum FileClass {
    File(Arc<OSFile>),
    Pipe(Arc<Pipe>),
    Socket(Arc<Socket>),
}

```



```

    FsContext(Arc<FsContextFd>),
    DetachedMount(Arc<DetachedMountFd>),
    Abs(Arc<dyn File>),
}

```

其中 File 用于普通 ext4 文件 ;Pipe 用于匿名管道与 FIFO 端点 ;Socket 用于网络套接字 ;Abs 用于设备、epoll、eventfd、inotify、signalfd、mqueue、io_uring 占位等抽象文件 ;FsContext 和 DetachedMount 支撑 Linux 新挂载 API 的 fd 流程。

8.3.2 FSInfo

FSInfo 保存进程文件系统环境：

```

pub struct FSInfo {
    inner: RwLock<FSInfoInner>,
}

struct FSInfoInner {
    cwd: String,
    exe: String,
    fd2path: HashMap<usize, String>,
    umask: u32,
}

```

- cwd 用于相对路径解析；
- exe 记录当前进程可执行文件路径；
- fd2path 辅助 /proc、dup 和调试场景维护 fd 到路径的映射；
- umask 默认 0o022，创建文件或目录时用 mode & !umask 计算最终权限。

8.4 路径解析与打开文件

sys_openat() 是用户态打开文件的主要入口。它从用户空间读取路径字符串，根据 dirfd 和进程 cwd 生成绝对路径，处理 /proc/self/* 等动态路径后调用 open(abs_path, flags, mode)；分配 fd 时写入 FdTable 并更新 FSInfo.fd2path。

open() 内部的查找采用分层缓存，尽量让热路径不进入 lwext4：

1. 先查 FsIndex inode 缓存（以 (st_dev, st_ino) 为身份键的路径索引）；
2. 未命中时，若父目录已缓存，通过 find_from_cached_parent() 只做末级目录项查找，并查询 DENTRY CACHE；
3. 仍未命中时回退到根 inode 的完整路径 find()，结果回填到 inode 索引和目录项缓存。

查找结果会缓存最终 inode；O_NOFOLLOW 与内部 O_UNLINK 保留末级符号链接本身，且不会把符号链接写入普通路径缓存，避免后续普通 open 复用错误的链接节点。负目录项（确认不存在）也会缓存，服务非 O_CREAT 的探测路径；创建路径会先失效或绕过 negative 项。

打开流程还会检查路径所在挂载的标志：NOSYMFOLLOW 会为普通 open 追加 O_NOFOLLOW，NODEV 禁止打开设备节点，只读挂载上的 O_CREAT 返回 EROFS。O_PATH 创建仅路径描述符，忽略创建、截断和访问模式；O_NOATIME 需要文件所有者或 CAP_FOWNER；创建新节点时按父目录权限、进程 umask 计算最终权限，并在父目录带 S_ISGID 时继承组 ID 与 setgid 位。以写模式打开既有普通文件时，还会向持有文件租约的进程发送租约断开通知。

/proc/uptime 与 /proc/<pid>/pagemap 是动态 proc 节点：前者按需合成开机时长，后者在读取时按地址空间实时生成页表项，避免为每个 fork 的进程分配数百 MiB 的 ext4 空洞文件。

O_TMPFILE 目前是一个内存匿名普通文件 (files/tmp_file.rs)：openat 只使用目标目录选择文件系统上下文，返回的 fd 持有内存中的数据和权限，直到关闭或通过 linkat("/proc/self/fd/<fd>") 落地为真实目录项，而不是在目录下创建 N.tmp。

8.5 目录项与 inode 缓存

FsIndex 是 VFS 层 inode 缓存，DENTRY_CACHE 是目录项缓存，二者分别加速“路径 → inode 对象”和“父目录 → 子 inode”的映射。

```
struct InodeCacheState {
    paths: HashMap<String, InodeCacheKey>, // 绝对路径 -> 身份键
    inodes: HashMap<InodeCacheKey, Arc<dyn Inode>>, // 身份键 -> inode 对象
}

enum InodeCacheKey {
    Inode { dev: usize, ino: usize }, // 正常文件的 (st_dev, st_ino)
    Path(String), // 无 inode 号后端的回退键
}
```

路径索引与 inode 缓存必须在同一次锁持有中更新，否则底层 ext4 在两步之间复用 inode 号时，新文件可能错误拿到已删除文件的 Arc<dyn Inode>。缓存上限为 32K 个 inode；SPECIAL_NODE_TYPES 表为 FIFO、设备节点等无法从后端稳定取得类型的节点保留创建时类型。

目录项缓存以 (父目录 inode 地址, 子项名) 为键，避免为构造键再次调用 path() 或 fstat()。缓存项分 Positive(保存子 inode) 和 Negative(确认不存在) 两类；create/link/unlink/symlink/rename 等命名空间操作显式回填或失效条目。缓存有界，达到上限和进程回收边界时统一清理。

这套分层缓存显著降低了深层路径的打开开销：父目录缓存命中时，open() 直接复用 FsIndex 中的父 inode 与 DENTRY_CACHE 中的子项，不再为每个目录层级重复进入 lwext4；末级 miss 时所有父级已完成解析，child 不存在无需再逐前缀扫描中间符号链接。

8.6 文件页缓存

os/src/fs/page_cache.rs 为普通文件的 read、mmap 和 splice 路径提供共享的页级缓存。缓存以 (文件路径, 页号) 为键保存页帧，帧直接使用 mm 的 Arc<FrameTracker>，因此文件映射与缺页路径复用同一物理页：

```
pub struct FilePageKey {
    pub path: Arc<str>, // 共享路径，零拷贝构造键
    pub page_index: usize, // 文件内以 PAGE_SIZE 为单位的页号
}

pub struct FilePage {
    pub key: FilePageKey,
    pub frame: Arc<FrameTracker>,
    pub valid_len: usize, // 尾页有效字节数
}
```

缓存容量由全局页数上限控制（默认 192K 页，约 768 MiB）。索引采用路径分组的两级结构，路径以 Arc<str> 跨 inode 别名共享，避免数十万次命中反复分配和复制 pathname。容量不足时使用带延迟队列的 CLOCK 策略回收未被引用的干净页：刚访问过的页、仍被其他对象引用的页和脏页不会被立即回收，固定页候选会在累计若干次容量失败后按小批次重试，避免 mmap 密集负载反复扫描整个工作集。顺序缺页最多预读 4 页（16 KiB），摊薄 lwext4 锁和单页读取开销；超过 32 MiB 的大文件只探测一次缓存准入，避免 mmap 触发重复的准入探测。

缓存只负责查找、加载、预读和失效,不负责把脏页写回文件;写入路径通过 `mark_dirty` 标记脏页,回写仍由 `lwext4` 与 `sync` 路径完成。文件映射的缺页通过 `FilePageCacheSource::MmapDemand` 与 `MmapPrefetch` 区分来源,命中后直接把缓存的 `FilePage` 帧映射到地址空间。

普通读路径与文件映射共用这些帧:大于一页的 `read()` 只对连续未命中的冷页段进入 `inode.read_at()`,命中页直接拼装到输出,避免单页 `miss` 导致整段回源重新读取;干净的只读私有文件页可跨进程复用同一缓存帧,通过 `COW` 保持私有写语义。

8.7 ext4 与 lwext4 集成

8.7.1 并发与资源锁

`ext4` 适配层早期使用挂载级的 `EXT4_OP_LOCK` 串行化所有文件系统操作。该锁竞争严重且会让无关操作互相等待;当前实现已把它替换为按资源分类的任务感知锁。C 侧 `struct ext4_fs_rwlock` 的地址即资源键,标识命名空间、inode 分片、块组分片、日志、超级块或缓存资源;Rust 侧为每个锁维护 FIFO 等待队列,发生竞争时任务睡眠在它实际需要的资源上,而不是在锁持有者被抢占时自旋。

```
struct TaskRwLockState {
    readers: BTreeMap<usize, TaskRwLockReader>, // 按任务记录读锁递归深度
    writer: Option<usize>,
    writer_depth: usize,
    next_ticket: usize,
    waiters: VecDeque<TaskRwLockWaiter>,        // FIFO 票号队列
}
```

锁模式按任务递归:同一任务可重入读锁(适配 `readlink` → `fread` 等 C 封装),写锁持有者可进入读区间;等待者按票号 FIFO 排队,已排队的写者之前新读者不能插队,避免读者饿死写者。任务退出时会通过 `cancel_ext4_op_waiter` 清理被抛弃的锁等待。两层之间的锁顺序固定为 `VFS write_state` → `VFS io_state` → `lwext4 resource locks`;C 层持锁时不回调 `VFS`,Rust 侧 `dentry`、`inode` 索引和页缓存锁持有时间很短,并在进入 `lwext4` 或发起块 I/O 前释放。块设备层 (`os/src/drivers/disk.rs`) 也采用任务感知的 FIFO 等待,减少 SMP 下块设备提交的竞争。

8.7.2 块设备适配

Ya2yOS 通过 `lwext4_rust` 将 `lwext4` 接入 Rust 内核。适配层向 `lwext4` 提供块设备接口:

```
pub trait BlockDevice {
    fn read(&self, buf: &mut [u8]) -> Result<i32, SysErrNo>;
    fn write(&self, buf: &[u8]) -> Result<i32, SysErrNo>;
    fn seek(&self, pos: usize);
    fn size(&self) -> usize;
}
```

内核的 `Disk` 结构实现该接口,使 `ext4` 可以直接在 `VirtIO` 块设备上执行目录项、inode、数据块和元数据操作。块设备层为 `lwext4` 提供任务感知的 FIFO 提交,并接入 `bcache` 与块请求遥测。

8.7.3 Ext4Inode

`Ext4Inode` 是 `Ext4File` 的 `VFS` 封装,主要能力包括:

- `create()/create_with_metadata()`: 创建普通文件或目录,并返回新的 `VFS inode`;
- `find()/find_from_cached_parent()`: 检查目录、普通文件和符号链接,符号链接支持绝对/相对目标;
- `read_at()/write_at()`: 打开底层 `ext4` 文件, `seek` 到指定偏移后读写;
- `read_all()`: 一次性读取普通文件内容,符号链接会递归读取目标;
- `read_dentry()`: 将 `lwext4` 目录项编码为 `getdents64` 可返回的数据;

- `truncate()`：通过 `lwext4` 截断或扩展文件，支持 `SEEK_DATA/SEEK_HOLE` 语义；
- `rename()/hard_link()/sym_link()/unlink()`：提供重命名、硬链接、软链接和删除；
- `fstat()/fmode()/fmode_set()/set_timestamps()`：返回和修改 Linux 兼容的元数据与时间戳；
- `set_xattr()/get_xattr()/list_xattr()/remove_xattr()`：提供真实扩展属性读写；
- `sync()`：刷新文件缓存；
- `delay()`：标记延迟删除，inode drop 时移除文件。

`Ext4Inode::fstat()` 会将 `lwext4` 返回的元数据转换为内核 `Kstat`，并对部分时间戳高位进行兼容性修正。`size()` 使用 `known_size` 快路径避免热路径重复进入 `lwext4`，`known_size` 在截断/写入等操作时失效；inode 类型 (`types()`) 走无锁只读字段，避免类型判断再次查询路径。

8.7.4 stat 与元数据缓存

目录的 `fstat/stat` 结果按目录本地元数据 epoch 缓存：一次 `pathname` lookup 取得的目录 `stat` 会被后续同目录查询复用，避免重复进入 `lwext4`。`mark_directory_stat_changed()` 钩子由命名空间操作 (`create/rename/rmdir` 等) 调用，推进受影响父目录的本地 epoch；`stat_cache_miss_reason` 记录每种 miss 的来源用于性能归因。缓存按目录局部失效，跨目录不互相干扰。

8.8 普通文件 IO

`OSFile` 封装一个普通 `ext4` inode，并维护当前文件偏移：

```
pub struct OSFile {
    readable: bool,
    writable: bool,
    pub inode: Arc<dyn Inode>,
    inner: Mutex<OSFileInner>,
}

struct OSFileInner {
    offset: usize,
}
```

`read()` 从当前 `offset` 调用 `inode.read_at()`，读到 EOF 返回 0，并推进 `offset`。大于一页的读请求会先尝试拼装文件页缓存中已驻留的页，只有未缓存的连续段才进入 `inode.read_at()`，避免单页 miss 导致整段重新读取并争用 `ext4` 锁。`write()` 调用 `inode.write_at()` 写入每个用户缓冲片段并推进 `offset`。`lseek()` 支持 `SEEK_SET`、`SEEK_CUR`、`SEEK_END`，并支持 `SEEK_DATA/SEEK_HOLE`（在 `lwext4` C 层实现）。`OSFile` 在 `open` 时缓存文件类型，`lseek()` 不再为每次调用重复查询路径与特殊节点表；FIFO/socket 的 `ESPIPE` 语义保持不变。

系统调用层为了避免超大用户请求导致内核堆 OOM 将 `read/write/readv/writev/pread64/pwrite64/sendfile/copy_file_range` 等操作按 64 KiB 上限分片或限制单次内核缓冲区大小。`fsync/fdatasync/sync_file_range` 最终调用 `inode` 的 `sync()`；当前不区分数据和元数据同步。

`sendfile()` 和 `copy_file_range()` 当前通过内核缓冲区在两个 `fd` 之间转发数据，并支持显式 `offset` 指针的读写回填，不是真正的零拷贝实现。

`fallocate()` 支持默认模式和 `FALLOC_FL_KEEP_SIZE`，会根据 `statfs` 的可用块数做空间检查；未支持的模式返回 `EOPNOTSUPP`。

8.8.1 写路径与写回缓存

写路径围绕 `lwext4` 的整文件写回缓存做专门优化。同一路径已打开的 `O_RDWR` 描述符可直接服务只读访问，避免 `O_RDWR -> O_RDONLY -> O_RDWR` 重开。写入命中已缓存且位于配额预留范围内时，以每 `inode` 的可睡眠状态锁串行，不再占用全局 `lwext4` gate；只有 `cache miss`、稀疏或超限写入、

延迟删除、淘汰回写才进入 lwext4 慢路径。整文件缓存采用有界 LRU，写命中提升活跃产物，避免并行编译的临时文件反复被驱逐、重建和整文件回写。写入前通过挂载表 reserve_write 预留字节配额，失败回滚，预留/回滚逻辑移出全局操作锁。稀疏写入以 (mount, inode) 有界 range 缓冲记录已提交的稀疏范围，读取时按范围覆盖 hole；rename 拆分 byte-cache 写回与 mount-wide flush，不再隐式同步整个挂载。

8.9 目录项与元数据

getdents64 通过 Inode::read_dentry(off, len) 读取目录项。ext4 层从 lwext4 获取目录项列表，并按用户缓冲区容量逐项编码返回，同时返回新的目录偏移。

Kstat 与 Linux stat 结构兼容：

```
pub struct Kstat {
    pub st_dev: usize,
    pub st_ino: usize,
    pub st_mode: u32,
    pub st_nlink: u32,
    pub st_uid: u32,
    pub st_gid: u32,
    pub st_rdev: usize,
    pub st_size: isize,
    pub st_blksize: i32,
    pub st_blocks: isize,
    pub st_atime: usize,
    pub st_atime_nsec: usize,
    pub st_mtime: usize,
    pub st_mtime_nsec: usize,
    pub st_ctime: usize,
    pub st_ctime_nsec: usize,
}
```

Statfs 返回文件系统块数、inode 数、文件名长度、挂载标志等统计信息。statfs() 当前直接返回全局 ext4 superblock 数据；fstatfs() 对 fd 做有效性检查后同样返回全局统计。

8.10 管道、FIFO 与 splice

8.10.1 Pipe

匿名管道由 Pipe 端点与共享 PipeRingBuffer 组成。缓冲区按字节容量限制保存 PipeBuf 片段队列，而不是早期版本的固定头尾环形数组：

```
pub struct Pipe {
    readable: bool,
    writable: bool,
    nonblocking: AtomicBool,
    buffer: Arc<Mutex<PipeRingBuffer>>,
}

pub(super) enum PipeBufStorage {
    Bytes(Arc<Vec<u8>>), // 普通 write 产生的数据
    FilePage(Arc<FilePage>), // 来自文件页缓存的帧引用
}
```

普通 write 直接以用户缓冲片段生成 Bytes 片段，不再先聚合临时 Vec；大读把片段直接写入用户页片段，同样避免聚合中间 Vec。空管道读会睡眠等待写入或写端关闭，满管道写会等待读端释放空间，非阻塞模式返回 EAGAIN；无读端时写返回 EPIPE 并发送 SIGPIPE。等待者通过弱引用任务队列与 PollSet 唤醒，避免在管道压力测试中忙等。管道容量可通过 F_SETPIPE_SZ 调整，受 CAP_SYS_RESOURCE 与全局 sysctl 上限约束。

8.10.2 FIFO 命名管道

FIFO 使用路径键控的共享环形缓冲表：open_fifo(path, flags) 按路径查找或创建缓冲区，读端/写端共享同一 PipeRingBuffer，并支持 O_NONBLOCK 下无读端时写打开返回 ENXIO。FIFO 端点是普通 Pipe 的变体，读/写/等待语义与匿名管道一致。

8.10.3 splice

splice 为 pipe → pipe 提供零拷贝快路径：直接移动 PipeBuf 片段本身 (Bytes 移动 Arc<Vec<u8>>, FilePage 移动 Arc<FilePage>)，两个 pipe 之间不复制片段内的实际数据；tee 通过克隆片段元数据保留输入 pipe 的数据。同时持有两个 pipe 缓冲时按 Arc 地址排序加锁，避免 splice/tee 形成锁序死锁。文件与 pipe 之间的 splice 复用文件页缓存，将缓存帧作为 FilePage 片段推入管道。

8.11 devfs、loop 设备与 proc 动态文件

8.11.1 devfs 与 loop 设备

devfs 维护一个设备路径到设备号的表，open() 识别设备路径后返回对应抽象文件。当前支持：

- /dev/null：读返回 EOF，写丢弃数据；
- /dev/zero：读返回全 0，写丢弃数据；
- /dev/random：提供伪随机/兼容性数据；
- /dev/rtc、/dev/rtc0、/dev/misc/rtc：返回简化 RTC 时间；
- /dev/tty：复用标准输入输出；
- /dev/cpu_dma_latency：记录 CPU 延迟配置；
- /dev/loop-control、/dev/loopN、/dev/loop/N、/dev/block/loopN：提供 loop 设备 ioctl 兼容。

loop 设备维护 256 项 LoopState，支持 LOOP_CTL_GET_FREE、LOOP_SET_FD、LOOP_CLR_FD、LOOP_GET_STATUS*、LOOP_SET_STATUS* 和 BLKGETSIZE64 等接口。当前 loop 块设备主要服务 LTP/BusyBox 兼容，尚未把数据读写真实转发到 backing file。

8.11.2 proc 动态文件

Ya2yOS 创建 /proc、/proc/mounts、/proc/meminfo、/proc/sys/kernel/* 等基础文件，并为每个进程创建 /proc/<pid>/stat、/proc/<pid>/status、/proc/<pid>/maps。这些文件的内容由内核生成后写入 ext4 文件；访问 /proc/self/* 或指定 pid 文件时会按需刷新。

/proc/uptime 是动态只读节点，按当前时钟与空闲 tick 实时合成；/proc/<pid>/pagemap 在读取时按地址空间逐页生成 PFN/PRESENT 位，避免为每个 fork 进程在 ext4 上分配与最高用户地址等大的稀疏文件。/proc/<pid>/status 提供 Uid/Gid 与真实进程组信息。

这不是完整的独立 procfs，而是一个以 ext4 文件承载内容、叠加动态节点的兼容实现，重点满足 libc、BusyBox 和 LTP 对常见 proc 节点的读取需求。

8.12 IO 多路复用与事件 fd

8.12.1 poll/select/epoll

普通文件、管道、设备、socket、eventfd、inotify、signalfd 等对象通过 `File::poll()` 暴露就绪状态。`ppoll()` 和 `pselect6()` 线性扫描用户传入的 fd 集合；`epoll_create1()` 创建 `EpollFile`，`epoll_ctl()` 管理监听集合，`epoll_pwait()` 返回就绪事件或阻塞等待。

epoll 文件对象内部维护：

- registry：fd 到监听事件的映射，与进程本地 fd 复用隔离，避免 fd 编号复用时串扰；
- ready_list：已触发的事件队列；
- PollSet：用于唤醒等待中的任务。

近期修复包括唤醒丢失（写入事件与任务睡眠之间的竞态）、stdin 与 eventfd 的 poll 就绪报告，以及 fd 分配预留与并发分配竞态。

8.12.2 eventfd

`eventfd2(initval, flags)` 创建一个 64 位计数器 fd，支持 `EFD_CLOEXEC`、`EFD_NONBLOCK`、`EFD_SEMAPHORE`：

- 普通模式读返回计数器值并清零；
- 信号量模式读返回 1 并将计数器减 1；
- 写入会累加计数器，写入 `u64::MAX` 返回 `EINVAL`；
- 计数器为 0 时阻塞读，计数器接近溢出时阻塞写；
- `poll()` 在计数器大于 0 时报告可读，在未满时报告可写。

8.12.3 signalfd、inotify、mqueue 与相关对象

`signalfd4()` 从用户掩码构造 `SignalFd`，读取时返回任务的 pending 信号队列；掩码剥离 `SIGKILL/SIGSTOP`，支持 `SFD_CLOEXEC/SFD_NONBLOCK`，并可用 `update_signal_mask()` 修改既有 fd 的掩码。`inotify_init1()` 创建 inotify fd，`inotify_add_watch()` 和 `inotify_rm_watch()` 管理 watch descriptor；当前 watch 管理、事件队列、read/poll 框架已经具备，但文件系统变更事件的自动生成仍是后续工作。`fanotify` 提供 `fanotify_init` 的 fd 载体、`fanotify_mark` 的 mark 表和覆盖 LTP `fanotify01` 的基础事件队列；权限事件响应与完整传播语义仍需接入。

POSIX `mqueue` 通过全局名称表管理队列，`mq_open()` 创建或打开命名队列，`mq_timedsend()` 和 `mq_timedreceive()` 发送/接收消息，支持非阻塞语义。`mq_notify()` 当前返回 `ENOSYS`。

`memfd_create()` 返回匿名内存文件 (`TmpFile`)，校验 `MFD_*` 标志并支持 `MFD_CLOEXEC`。`io_uring_setup()` 返回独立的 `IoUringFd` 占位 fd，并提供真实的 SQ/CQ ring offsets 与 `IORING_MAX_ENTRIES` ABI 结构，但提交/完成环引擎尚未实现。`timerfd_create`、`memfd_secret`、`perf_event_open` 等仍返回 dummy fd 或 `ENOSYS`，用于避免用户程序在能力探测阶段直接失败。

8.13 挂载接口

`MNT_TABLE` 是当前挂载语义的状态中心。每个 `MountEntry` 保存 (`special`, `dir`, `fstype`, `flags`) 以及 `shared_group`、`master_group`、`unbindable`、`event_group`。表最多容纳 256 个条目，允许同一路径出现多个挂载层；按路径查询时选择最长覆盖路径，同一挂载点选择最新层，因此 `umount2()` 删除顶层后会重新显露更早的层。`/proc/mounts` 由该表序列化，根 ext4 记录始终存在。

传统 `mount()` 支持普通挂载、`remount`、`MS_BIND`、`MS_MOVE` 和 `propagation-only` 调用。`MS_SHARED` 为选定挂载副本建立 peer group，`MS_SLAVE` 保留上游 master 关系，`MS_PRIVATE/MS_UNBINDABLE` 断开传播关系，`MS_REC` 可将这些属性递归应用到子树。在 `shared` 挂载或其 slave 后代下创建子挂载时，表会按相对路径扩展副本：事件从 peer 流向 slave，但不会由 slave 反向传播到 master。`MS_UNBINDABLE` 源不能被 bind clone，bind 事件会经 peer/slave 传播，`MS_MOVE` 移动整棵子树并清除源挂载。

MS_MOVE 移动整棵挂载子树，并为目标父挂载可达的 peer/slave 创建副本；每次普通 挂载或移动产生一个 event_group，卸载任何一个副本时会同时删除同组副本。bind/move 的系统调用层还会在表锁外镜像目录树：目录递归创建，普通文件使用 hard link。这使当前 路径式 VFS 能观察到常见 bind/传播测试的目录形状，同时避免在递归 bind 中复制目标自身。

fsopen、fsconfig、fsmount、fspick、open_tree、move_mount 和 mount_setattr 已提供 fd 类型、参数校验与最小状态流。fsconfig 记录选项并在 create 后允许 fsmount 生成 detached mount fd；move_mount 可将它落入 MNT_TABLE。这些 接口尚未创建独立 superblock 或 namespace，mount_setattr 目前主要校验 ABI 结构。fresh tmpfs 挂载会清空底层挂载点目录，以在该简化模型中近似空的 tmpfs 根目录。跨进程仍持有打开的 inode 时，unlink 会延迟到底层引用释放后执行，避免删除后仍可通过 fd 访问的路径被缓存污染。挂载表还为每个挂载维护模拟配额：reserve_write 在 写前预留字节配额，失败回滚，容量由 loop 设备格式化镜像大小推导。

8.14 文件系统锁设计

文件系统是当前内核中并发最密集的子系统之一。锁按 VFS 层 → ext4 适配层 → lwext4 C 资源锁 → 块设备 分层组织，设计目标是把竞争收敛到真正需要的资源粒度上，而不是让整个文件系统或单个挂载成为单一临界区。

8.14.1 VFS 层锁

VFS 对象锁保护各缓存与描述符的短期状态，锁持有时间都很短，且不跨越文件系统 I/O、调度或信号路径：

- FsIndex 与 DENTRY_CACHE 用 RwLock 保护路径索引与目录项缓存，路径到 inode 的映射在同一次锁持有中更新，避免 inode 号复用被观察到中间态；
- 文件页缓存 FILE_PAGE_CACHE 用 RwLock 保护两级索引，驱逐队列用独立 Mutex 保护；
- OSFile 的偏移与可变状态用 Mutex，管道共享缓冲用 Arc<Mutex<PipeRingBuffer>>；
- FdTable、FSInfo 分别用 RwLock 保护进程级描述符表与文件系统环境，MNT_TABLE 用 Mutex 保护挂载表。

这些锁在进入 lwext4 或发起块 I/O 前释放，因此不会与底层锁嵌套。

8.14.2 任务感知锁

ext4 适配层把 C 侧锁钩子接到任务感知的 TaskMutex/TaskRwLock：发生竞争时调用者睡眠在它实际需要的资源上，而不是在锁持有者被抢占时自旋；没有当前任务（启动阶段）的调用者保留自旋回退。锁按任务递归——同一任务可重入读锁（适配 readlink → fread 等 C 封装），写锁持有者可进入读区间；等待者按票号 FIFO 排队，已排队的写者之前新读者不能插队，避免读者饿死写者。任务退出时通过 cancel_ext4_op_waiter 清理被抛弃的锁等待。C 侧 struct ext4_fs_rwlock 的地址即资源键，标识命名空间、inode 分片、块组分片、日志、超级块或缓存资源，具体分片与锁序详见第九章。

8.14.3 锁序与内存衔接

两层之间的锁顺序固定为 VFS write_state -> VFS io_state -> lwext4 resource locks；C 侧资源锁之间为 journal_lock -> cache_flush_lock -> cache_lock，超级块计数始终在 super_lock 下更新。C 层持锁时不回调 VFS。块设备层 (os/src/drivers/disk.rs) 也用任务感知的 FIFO 提交队列：单一同步 VirtIO 队列要求请求独占通过对齐批量或未对齐 RMW，任务用 block_on 挂起而不是自旋，任务退出时 cancel_disk_waiter 清理票号。

与内存管理的衔接遵循既有约定：页表更新遵守 UPDATE_LOCK -> MemorySet 写锁，跨地址空间操作先快照 Arc 帧再切换锁；进入文件系统、调度、网络或信号路径前释放 MemorySet guard。文

件缺页、fork 预取与共享页写回均做“锁外 I/O、锁内短暂页表更新”，避免把可睡眠的文件系统锁嵌套进内存锁，也避免在持有内存锁时进入 lwext4。

8.15 文件锁、fcntl 与 xattr

fcntl() 支持 fd 复制、FD_CLOEXEC、O_NONBLOCK 查询/设置、pipe 大小查询、文件 owner 相关兼容返回，以及 POSIX record lock / OFD lock 的基本语义。F_SETLEASE/F_GETLEASE 提供文件租约，F_SETOWN/F_SETSIG 等配置异步 I/O 信号目标。fchdir 与 fchmodat2 提供基于 fd 与 AT_* 标志的目录/权限操作。

文件锁实现分三类：

- flock()：按打开文件描述（而非路径）维护整文件 advisory lock，支持共享锁、排他锁、非阻塞失败和阻塞等待唤醒；
- F_GETLK/F_SETLK/F_SETLKW：按 inode 路径维护字节区间锁，F_SETLKW 在等待图中做死锁检测；OFD lock 按打开文件描述隔离，委托给同一套记录锁逻辑；
- F_SETLEASE：文件租约在冲突打开时通过 SIGIO 通知持有者，由 open 路径调用 notify_file_lease_break()。

扩展属性已有真实的 syscall 与文件系统路径：setxattr/getxattr/listxattr/removexattr 经 os/src/syscall/fs/xattr.rs 解析用户指针后调用 Inode::*_xattr，由 lwext4 在文件系统操作锁内完成 XATTR_CREATE/XATTR_REPLACE 校验与数据读写。

8.16 动态链接支持

map_dynamic_link 模块负责兼容用户程序期望的动态链接器和共享库路径。open() 和 openat() 会在进入 ext4 前做路径映射；read_at() 和 read_all() 会通过 patch_dynamic_link_file_bytes() 对部分动态链接文件内容做运行时修补。

这套机制让固定镜像中的 musl/glibc 程序可以在内核统一的文件系统布局上运行，同时避免把所有路径兼容都硬编码到用户态。链接器从原生 libc script 显式命名解释器时优先使用该解释器，镜像缺失时才回退到兼容目标路径。

8.17 当前边界与后续方向

1. 真实多文件系统挂载：挂载表已支持叠加层、bind/move 子树和 shared/slave 传播，但路径解析仍没有挂载点 inode 切换、独立 superblock 或 mount namespace；bind 目录可见性依赖目录镜像而非 VFS dentry 切换。
2. **tmpfs/devtmpfs/procfs**：/dev 与 /proc 目前是兼容实现。后续可将它们提升为独立内存文件系统，减少对 ext4 承载虚拟文件内容的依赖。
3. **loop** 设备数据路径：loop ioctl 状态管理已实现，但块读写尚未转发到 backing file。
4. **inotify** 事件生产：watch 管理和 fd 读写框架已经具备，仍需在 create/unlink/rename/write/chmod 等 VFS 操作中插入事件生成钩子；fanotify 的权限事件响应与完整传播语义仍未接入。
5. 异步 IO：io_uring_setup 提供占位 fd 与 ring offsets ABI，但提交/完成环引擎尚未实现；完整 io_uring/AIO 数据路径仍需结合统一缓存和 poll 唤醒机制继续扩展。
6. 锁语义补全：POSIX record lock 的等待图死锁检测、OFD lock 与文件租约已实现；close 时自动释放锁与更贴近 Linux 的租约生命周期仍可继续完善。
7. **xattr** 持久化：xattr 已有真实读写路径，但其持久化存储与一致性仍需进一步验证。

实现追溯：本章对应 os/src/fs/ (vfs.rs、fstruct.rs、fs_info.rs、dcache.rs、page_cache.rs、mount.rs、stat.rs、map_dynamic_link.rs、ext4_lw/、kernel_fs_ops/、files/)、os/src/syscall/fs/ 以及 os/src/drivers/disk.rs 的当前实现。

第九章 lwext4 与磁盘文件系统引擎

9.1 概述与定位

crates/lwext4_rust 是 Ya2yOS 的磁盘文件系统引擎，由 C 实现的上游 lwext4 库与 Rust 绑定层组成。上游 lwext4 是一个面向微控制器的 ext2/ext3/ext4 文件系统库 (BSD 类许可证，当前 fork 采用 GPL-2.0)，这里在其上叠加了面向 SMP 内核的任务感知锁、块缓存并发与写回缓存等大量定制。第八章的 os/src/fs/ext4_lw/ 通过 lwext4_rust 暴露的接口把 VFS 操作落到磁盘格式。

crate 的模块结构如下 (lib.rs)：

```
mod ulibc;
pub mod bindings;
pub mod blockdev;
pub mod file;
pub mod perf;

pub use blockdev::*;
pub use file::{Ext4File, InodeTypes};
```

- bindings.rs：由 c/wrapper.h 经 bindgen 生成的 C 结构、常量和函数绑定；
- blockdev.rs：Ext4BlockWrapper 与 KernelDevOp trait，封装 ext4_blockdev 与块缓存，处理挂载/日志生命周期；
- file.rs：Ext4File，封装 C 侧文件描述符，并提供整文件写回缓存与稀疏写缓冲；
- perf.rs：可选性能遥测，聚合块缓存与写回缓存计数器；
- ulibc.rs：为无标准库的 C 代码提供 printf、malloc 等弱符号。

Cargo feature 控制三组能力：print 把 C 的 printf 路由到 Rust 的 printf-compat 兼容实现；perf 同时开启 C 侧 EXT4_PERF_TELEMETRY 与 Rust 侧计数器；bcache-dirty-capacity-experiment 开启脏块缓存高低水位回收实验。

9.2 构建与绑定

C 静态库由 build.rs 驱动 CMake 按目标架构编译。上游源码以子模块形式位于 c/lwext4/，构建时应用 c/lwext4-make.patch 并注入 crate 自带的 musl-generic.cmake 工具链：

- 按 CARGO_CFG_TARGET_ARCH 使用 <arch>-linux-musl-* 工具链，-ffreestanding -fPIC -fno-builtin，静态归档命名为 liblwext4-{arch}-p212b2-default-{no-perf-}c2048.a；
- 归档名编码 feature 变体与 LWEXT4_BLOCK_CACHE_SIZE=2048，并比较 C 源码与归档 mtime 决定是否重建，避免陈旧 Docker 产物被误复用；
- CMake 选项包含 LWEXT4_USE_USER_MALLOC (走 Rust 分配器)、EXT4_PERF_TELEMETRY 与 EXT4_BCACHE_DIRTY_CAPACITY_EXPERIMENT；
- x86_64 使用仓库提交的 bindings.rs，其他架构由 bindgen 基于 musl sysroot 重新生成。

由于 crate 是 #![no_std]，C 代码依赖的 libc 函数由 ulibc.rs 与 c/ulibc.c 提供弱符号回退：printf 走 printf-compat，malloc/calloc/realloc/free 通过带魔数和大小校验的 MemoryControlBlock 头交给 Rust alloc，并补充 strcpy/strcmp/memset/qsort_r 等。

9.3 磁盘布局模型

C 层的磁盘格式处理遵循标准 ext4 布局，结构全部 #pragma pack(1) 并手工小端转换：

- **superblock** (struct ext4_sbblock)：完整 ext4 超级块，含 64 位块计数、日志 inode 号与 UUID、哈希种子、crc32c 校验和；

- 块组描述符 (struct ext4_bgroup) : 块/ inode 位图地址、inode 表首块、空闲计数、BLOCK_UNINIT/INODE_UNINIT 标志与校验和，支持 64 位描述符；
- **inode** (struct ext4_inode) : 模式、uid/gid、64 位大小、带 nsec/epoch 的时间戳、blocks[15] (12 个直接块 + 3 个间接指针，未用 extent 时作为 extent 根)、file_acl (xattr 块指针) 与额外 isize；
- 目录：线性目录项 ext4_dir_en 与 htree 结构 (根/索引/叶节点)；
- **xattr** : inode 内嵌体 (ibody) 与 file_acl 指向的外部块两种位置，带 hash 与 crc32c 校验。

以 inode 为例，bindgen 生成的 ext4_inode 表示如下 (blocks[15] 在启用 extent 时保存 extent 根，否则保存 12 个直接块与 3 个间接指针)：

```
#[repr(C)]
pub struct ext4_inode {
    pub mode: u16,
    pub uid: u16,
    pub size_lo: u32,
    pub access_time: u32,
    pub modification_time: u32,
    pub gid: u16,
    pub links_count: u16,
    pub blocks_count_lo: u32,
    pub flags: u32,
    pub blocks: [u32; 15],      // extent 根, 或 12 直接 + 3 间接指针
    pub generation: u32,
    pub file_acl_lo: u32,      // xattr 外部块
    pub size_hi: u32,
    pub extra_isize: u16,
    pub checksum_hi: u16,
    pub ctime_extra: u32,
    pub mtime_extra: u32,
    pub atime_extra: u32,
}
```

feature 集支持 EXT4_FINCOM_FILETYPE | META_BG | EXTENTS | FLEX_BG | 64BIT 与只读兼容的 SPARSE_SUPER | METADATA_CSUM | LARGE_FILE | GDT_CSUM | DIR_NLINK | EXTRA_ISIZE | HUGE_FILE。CONFIG_JOURNALING_ENABLE、CONFIG_XATTR_ENABLE、CONFIG_EXTENTS_ENABLE 均开启，块缓存大小由 CMake 覆盖为 2048。

9.4 块缓存 bcache

块缓存用两棵红黑树维护：按 LBA 排序的 lba_root 与按 LRU id 排序的 lru_root，外加一个脏块单链表。每个缓冲区用原子位标志描述状态：BC_UPTODATE、BC_DIRTY、BC_FLUSH、BC_LOADING、BC_WRITEBACK、BC_EVICTING、BC_IO_ERROR，所有标志操作走 __atomic_*。

读路径采用单 loader 语义：ext4_block_get 命中时直接返回；miss 时用 CAS 抢占 BC_LOADING 位，获胜者发起物理读 (ext4_blocks_get_direct)，并发调用者调用 ext4_bcache_wait_while 等待——该等待会分发到宿主内核的 wait/wake 钩子 (见下文)，否则退化为 CPU 松弛。I/O 失败设置 BC_IO_ERROR，迟到的等待者拿到 EIO。ext4_bcache_retain 在索引锁内安全增加引用计数，修复了上游 refctr++ 的 SMP 竞态。

写回由 ext4_block_cache_flush 驱动：用 ext4_bcache_claim_dirty 认领脏块，把连续 LBA 合并成最多 32 个块的批量请求 (EXT4_BLOCK_CACHE_FLUSH_BATCH_MAX_BLOCKS)。

ext4_block_cache_write_back 是挂载级嵌套计数器，只在最外层 (on_off=0) 真正排水，排水过程锁定顺序固定为 journal_lock → cache_flush_lock → cache_lock。

单个缓存缓冲区的 bindgen 表示为：

```
#[repr(C)]
pub struct ext4_buf {
    pub flags: c_int,    // BC_* 原子位标志
    pub lba: u64,
    pub data: *mut u8,
    pub lru_id: u32,
    pub refctr: u32,     // 在 index_lock 内安全递增
    pub bc: *mut ext4_bcache,
    pub on_dirty_list: bool,
    pub end_write: Option<unsafe extern "C" fn(...)>, // 磁盘写完成回调
}
```

9.5 日志与事务

日志 (JBD) 使用 inode 8 作为日志 inode，维护一个循环日志：jbd_journal 保存日志边界与事务 id，jbd_trans 保存单事务的脏缓冲区队列、revoke 树与块记录。元数据块通过 ext4_trans_set_block_dirty 进入事务，同时持有 bcache 引用并安装 end_write 回调。

提交路径为：分配 trans_id，写 descriptor 块与带块标签的数据拷贝 (magic 与 JBD_MAGIC 相同时做转义)，写 revoke 块与 commit 块，然后进入检查点队列。检查点按每个缓冲区的写完成回调推进，written_cnt == data_cnt 时日志起点前移并释放事务；jbd_journal_make_room 始终保留一个日志槽位，避免日志写满时的自旋断言。

恢复由 jbd_recover 对日志做三遍扫描：ACTION_SCAN 定位日志起止事务，ACTION_REVOKE 从 revoke 块构造撤销树，ACTION_RECOVER 把日志块重放到文件系统位置 (跳过被新 revoke 覆盖的块)。恢复完成后清除 EXT4_FINCOM_RECOVER 并在 super_lock 下重新扫描所有块组计算空闲计数。

事务支持递归：ext4_trans_start/stop/abort 用嵌套计数，只有最外层才创建/提交共享的 curr_trans，内层失败设置 abort pending 由外层边界统一中止，并用 ext4_fs_rwlock_write_owned_by_current 校验持有权。

9.6 文件操作流程

挂载与卸载：lwext4_mount 依次执行 ext4_device_register → ext4_mount → ext4_recover → ext4_journal_start → ext4_cache_write_back(mp, true)；lwext4_umount 逆序回收。块设备回调 dev_open/dev_bread/dev_bwrite/dev_close 把 blk_id × ph_bsize 换算成字节偏移，要求完整长度 (短读写返回 EIO)。

目录与路径：ext4_dir_find_entry/add_entry 优先走 htree：哈希名字后在索引/叶节点二分，叶内线性扫描并处理哈希碰撞链；树损坏时清除 INDEX inode 标志并回退到跨块线性扫描/插入。哈希函数支持 Linux 的 Half-MD4、TEA 与 legacy 三种。

数据路径：ext4_fread 在 inode 读锁下逐块解析，快速符号链接直接读 inode 内嵌数据，物理连续的非零块合并为直接块读，hole 与 unwritten extent 零填充。ext4_fwrite 在写锁与事务内分配/追加块，批量数据用直接 IO 写入；payload 与尾零填充都绕过 bcache，避免脏缓存别名。稀疏写保留目标逻辑块超过 EOF 的部分。

extent 树：根在 inode 的 blocks 区域。ext4_extent_get_blocks 是读写共用的映射入口：返回覆盖块的已有 extent、写时把 unwritten 区间转为 initialized (直接写零填充后转换)、或按目标块

分配新块并插入。插入支持叶内前后合并、节点分裂、树加深 (ext4_ext_grow_indepth) 与索引修正；截断删除支持区间移除、叶中间拆分与整树高度收缩。

分配：块分配 ext4_balloc_alloc_block 目标位优先，其次按 64 位窗口与组轮转，空闲计数在 super_lock 下更新；释放块时写 revoke 记录并 ext4_bcache_invalidate_lba 丢弃缓存的已释放块。inode 分配 ext4_ialloc_alloc_inode 从 last_inode_bg_id 起扫描各组，优先空闲空间多的组。块位图与 inode 位图在 BLOCK_UNINIT/INODE_UNINIT 时惰性初始化，均在事务内完成。

rename/truncate：rename 支持替换既有文件 (ext4_frename)，并已修复替换时旧文件的生命周期；truncate 通过 extent 树收缩与 inode 尺寸更新，稀疏扩展保持逻辑块语义。

9.7 SMP 锁钩子

这是 Ya2yOS 对 C 层最核心的定制。上游 lwext4 用单个挂载级 EXT4_OP_LOCK 串行化所有路径操作，这里被替换为一组按资源分类的条纹读写锁：

- namespace_lock：路径名与命名空间变更；
- inode_locks[257] 与 group_locks[257]：按 inode % 257 / bgid % 257 分片；
- super_lock：超级块计数；
- journal_lock：日志事务；
- cache_lock 与 cache_flush_lock：写回缓存模式状态与排水。

每个资源锁的 C 结构只保存三类标量状态：

```
struct ext4_fs_rwlock {
    int state; /* 资源锁状态 */
    uint32_t writer_depth; /* 无钩子回退时写者递归深度 */
    uint8_t kind; /* 资源类别，供宿主内核分类 */
};
```

struct ext4_fs_rwlock 保存 state / writer_depth / kind，kind 供宿主内核分类锁（例如避免 timer 扫描死锁）。C 侧通过 ext4_fs_rwlock_set_hooks(ctx, lock_hook, unlock_hook, write_owned_hook) 暴露可安装钩子：安装后每个 lock/unlock 都分发给宿主内核，由 os/src/fs/ext4_lw/ 的任务感知 TaskRwLock 把等待者 park 在真正需要的资源上；无钩子（独立/宿主编译）时回退到原子自旋读写锁，允许写者通过 writer_depth 递归。ext4_fs_rwlock_write_owned_by_current 供事务助手检查当前任务是否持有写侧。

分片锁的覆盖范围与资源粒度对应：inode_locks[257] 按 inode % 257 分片，使不同 inode 的元数据操作并行，只有共享 inode 表块的读者互斥；group_locks[257] 按 bgid % 257 分片，使不同块组的位图与描述符更新互不阻塞。namespace_lock 串行化路径名与命名空间变更；super_lock 保护超级块空闲计数；journal_lock 保护日志事务；cache_lock 与 cache_flush_lock 保护写回缓存模式状态与排水。事务与锁交互遵循递归语义：ext4_trans_start/stop/abort 用嵌套计数，只有最外层创建/提交共享的 curr_trans，内层失败设置 abort pending 由外层边界统一中止；写侧归属用 ext4_fs_rwlock_write_owned_by_current 校验。完整锁序为 journal_lock → cache_flush_lock → cache_lock——cache flush 回调可能调用 journal 回调，该顺序保证检查点与缓存的交互串行化；超级块计数始终在 super_lock 下更新。

块缓存同样暴露 ext4_bcache_setup_sync(ctx, wait, wake)：一个 hart 加载/写回缓冲时，其他 hart 通过 wait/wake 钩子睡眠而不是自旋；index_lock 只保护 LBA/LRU 树与引用计数，且绝不跨越分配、I/O 或回调持有。缓冲加载采用单 loader：ext4_block_get miss 时用 CAS 抢占 BC_LOADING 位，获胜者直接读盘，其余调用者在 ext4_bcache_wait_while 上等待；写回由 BC_WRITEBACK 位与认领/释放机制串行。

Rust 侧写回缓存条目使用 VFileCacheLock（宿主锁钩子优先、spin::RwLock 回退）加独立的 flush 锁：write_back_cache_entry 先取 flush 锁快照缓存，释放后再按路径 0_RDWR 写回，写回后

用 revision 校验避免并发写者丢失脏数据；条目的锁在驱逐时由 release 钩子归还，避免为每个缓存路径泄漏一个调度器锁。

9.8 Ya2yOS 定制与写回缓存

在 C 层之上，Rust 侧维护一套整文件写回缓存与稀疏写缓冲，服务于编译器等大量小文件、稀疏文件写入的工作负载。设备接口由 KernelDevOp trait 抽象，要求位置无关的请求，保证 SMP 下块回调可并发：

```
pub trait KernelDevOp {
    type DevType;
    fn device_size(dev: &Self::DevType) -> Result<u64, i32>;
    fn read_at(dev: &Self::DevType, offset: u64, buf: &mut [u8]) ->
    Result<usize, i32>;
    fn write_at(dev: &Self::DevType, offset: u64, buf: &[u8]) ->
    Result<usize, i32>;
    fn flush(dev: &Self::DevType) -> Result<usize, i32>;
}
```

在此基础上维护以下状态：

- 写回缓存 CACHE_TABLE 按路径保存 Arc<VFileCacheLock>，FIFO_TABLE 是 32 项有界 LRU 写回队列；write_back_cache_entry 快照缓存、以 0_RDWR 重开路径写回，并用 revision 校验避免并发写者丢失脏数据；条目上限 4 MiB，对应 32 MiB 日志的提交空间；
- 稀疏写缓冲以 (mount, inode) 为键保存有界 range 集合（512 KiB / 32 段 / 8 MiB 全局预算），相邻与重叠写入先合并再提交，避免为每个子页写入展开 hole；
- SEEK_DATA/SEEK_HOLE 通过定制的 ext4_fseek_data/hole 实现；xattr 由 99947f41 补齐到 syscall 路径；
- 性能遥测分两层：C 侧 BcachePerfStats（命中/miss、loader、wait/wake、驱逐、写回、读写提交等约 34 项原子计数）与 Rust 侧 WriteBackCachePerfStats（缓存命中、直接写分类、稀疏写 flush 归因、rename/fstat 分阶段等约 60 项）；crate 只维护 relaxed 计数器，由内核 os::utils::perf 读取快照并补充锁与阶段计时。

9.9 当前边界

1. 日志空间约束：整文件写回缓存条目上限按 32 MiB 日志空间设定；密集写回需为 descriptor/ revoke 块与并行编译任务的检查点事务预留空间。
2. 块缓存容量：LWEXT4_BLOCK_CACHE_SIZE=2048 是当前镜像的固定配置；脏块高低水位回收（256/128）由 bcache-dirty-capacity-experiment feature 开启，默认构建不启用。
3. 挂载模型：当前为单挂载点/单设备模型（ext4_fs 挂在 /），未实现多 superblock 或挂载 namespace。
4. 一致性策略：payload 数据与尾零填充走直接 IO 绕过 bcache，依赖 C 侧单 loader 与写回排水保证与缓存元数据的一致性；脏块别名依赖 invalidate_lba 与 revoke 清理。
5. C 代码可移植性：部分定制（条纹锁、直接 IO、遥测）是 Ya2yOS 专属，独立/宿主编译会退化为无钩子自旋路径，磁盘格式处理与上游保持一致。

实现追溯：本章对应 crates/lwext4_rust/ (src/{lib,blockdev,file,perf,ulibc,bindings}.rs、build.rs、c/lwext4/src/) 以及 os/src/fs/ext4_lw/ 的当前实现。

第十章 AI 的使用情况

10.1 使用概况

在 Ya2yOS 的开发过程中，AI 工具的使用方式经历了从辅助问答到智能体协作的变化。项目早期主要通过网页版对话咨询操作系统设计思路、Linux 语义和局部代码实现；后期逐步引入 Claude Code、Codex 等智能体工具，让 AI 直接参与代码阅读、补丁编写、日志分析和回归验证。

本项目使用过的模型主要包括 GPT 系列、DeepSeek 系列和 Gemini Flash Preview 等。关于 AI 参与过程中的具体交互、分析路径、修改文件和验证结果，已在 Docs/决赛文档/ai.log 与 Docs/决赛文档/AI_INTERACTION.md 中持续记录。本章主要总结 AI 在项目中的作用、产出、局限以及由此带来的个人反思。

10.2 主要工作成果

在整个开发阶段，项目目标始终以本人学习和掌握内核开发为主。AI 则承担了大量重复性、探索性和工程性的工作，尤其集中在系统调用语义完善、内核调试、代码重构和文档整理等方面。

10.2.1 系统调用与兼容性补齐

AI 参与了大量 Linux syscall 的实现和语义修正工作，包括进程管理、文件系统、网络、信号、时间、权限和内存管理等模块。对于 LTP、libc-test、BusyBox、netperf、lmbench 等测试暴露的问题，AI 能够根据失败日志和内核代码快速定位相关路径，给出补丁草案，并在多轮验证中逐步收敛到可用实现。

这类工作对项目推进帮助较大。一方面，AI 能快速检索和对照 Linux 行为，补齐错误码、边界条件、结构体字段和兼容 stub；另一方面，它也能承担大量机械性的接入工作，例如新增 syscall 分发、补充参数检查、调整返回值和更新相关文档。

10.2.2 调试与问题复盘

内核开发中的许多问题并不是单点 bug，而是涉及任务调度、用户指针、页表权限、文件描述符、信号中断、网络状态机等多个模块。AI 在阅读日志、梳理调用链和提出排查方向方面提供了明显帮助，尤其适合处理 panic backtrace、LTP 失败输出和长时间阻塞等问题。

在修复完成后，AI 还辅助整理了 problem 文档，将问题背景、现象、分析过程、根因、修复方式和验证结果记录下来。这些文档降低了后续回看和维护的成本，也帮助本人从具体 bug 中反向理解内核机制。

10.2.3 代码审查与重构辅助

AI 生成的补丁并不会直接被接受。每次修改后，本人都会对代码进行审查，重点检查是否符合项目结构、是否引入冗余逻辑、是否破坏已有语义、是否存在过度特判以及是否影响其他测试路径。对于不符合要求的实现，会要求 AI 继续修改或重新给出更小范围的方案。

在重构类任务中，AI 能较好地完成跨文件重命名、重复逻辑合并、错误处理统一和注释补充等工作。对于原本较难理解的代码，AI 生成的解释性注释和变更说明也帮助本人更快掌握相关模块的设计意图。

10.3 使用中的不足

AI 的主要不足在于缺少对项目全局取舍的稳定判断。它能够基于局部日志和局部代码提出修改，但在问题边界不清晰时，容易扩大排查范围，对已经相对稳定、出错概率较低的代码反复审查，甚至在缺乏证据的情况下尝试修改无关路径。

以 netperf 相关功能调试为例，在没有人工干预时，AI 曾经表现出一定的盲目性：它会反复检查已经构建完成且短期内不太可能出问题的基础代码，消耗较多时间，却没有直接推进问题修复。只有在人工明确提供现象、限定怀疑范围、指出可能的触发条件后，AI 才能更有效地围绕关键路径展开分析，并给出更接近根因的修复方案。

此外，AI 生成的代码有时会出现以下问题：

1. 为了让单个测试通过而引入过度特判；
2. 对 Linux 语义理解不完整，尤其是错误码、阻塞语义和信号中断语义；
3. 在复杂并发或生命周期问题上给出看似合理但缺少不变量证明的修改；
4. 文档表达有时过于冗长，需要人工压缩为适合项目文档的形式。

因此，AI 更适合作为高效率的工程助手，而不是可以完全替代人工判断的开发主体。

10.4 个人反思

不可否认，当前项目中相当一部分新增代码由 AI 辅助生成。但这些代码最终是否进入项目，仍然经过了人工审查、测试验证和多轮修改。AI 提高了编码、调试和文档整理效率，也让本人能够在较短时间内接触更多真实内核问题。

对本人而言，AI 的价值并不只体现在生成代码上，更体现在学习过程本身。通过审查 AI 的补丁、比较不同修复方案、理解 problem 文档中的根因分析，本人逐步加深了对 Linux ABI、Rust 内核工程、文件系统、信号、网络 and 内存管理等模块的理解。

后续如果继续使用 AI 参与 Ya2yOS 开发，需要保持明确的协作边界：由人工提出目标、判断方向、审查设计并负责最终取舍；由 AI 承担资料整理、代码草拟、日志分析、测试反馈和文档归纳。只有在这种模式下，AI 才能真正服务于项目质量和个人能力提升，而不是把开发过程变成不可控的试错。

第十一章 总结与展望

11.1 项目成果总结

Ya2yOS 是一个基于 Rust 的宏内核实验操作系统，面向 Linux 用户态兼容和双架构运行环境持续演进。当前内核已贯通启动、进程调度、虚拟内存、文件系统、网络、信号到设备驱动的主要用户态路径，可以运行 BusyBox、libc-test、LTP 子集等负载；但若干关键边界仍是兼容实现、轮询路径或尚未完成的语义，不能将接口覆盖等同于完整 Linux 能力。

11.1.1 系统完整性

内核覆盖了以下关键子系统：

- 进程/线程、fork/clone/exec/wait、进程组和基础资源限制；
- Sv39/LoongArch 页表、mmap/brk、COW、文件映射和缺页处理；
- ext4 文件系统、VFS 抽象、fd 表、管道、设备文件、proc 兼容文件；
- TCP/UDP/Unix socket、poll/select/epoll、eventfd、inotify 框架和消息队列；
- 信号投递、信号帧、sigreturn、备选信号栈和常见 signal syscall；
- VirtIO 块设备、VirtIO 网络设备、RISC-V64 MMIO 与 LoongArch64 PCI 支持。

11.1.2 Linux 兼容性

Ya2yOS 实现了大量 Linux syscall，并围绕 glibc、musl、BusyBox、libc-test 和 LTP 进行了兼容性补齐。实现策略不是一次性追求完整 Linux，而是优先保证常见用户态路径可运行：对核心 syscall 提供真实语义，对低频或探测型接口提供明确的兼容 stub。

11.1.3 技术深度

当前较有代表性的实现包括：

- **COW 与 mmap**：覆盖 fork 后私有页、文件映射、脏页写回和懒分配等场景；
- 信号机制：支持信号帧、sigreturn、SA_SIGINFO、SA_RESTART、备选信号栈等关键语义；
- **ext4 集成**：通过 lwext4_rust 将 lwext4 接入内核，提供普通文件、目录、软硬链接、truncate、stat 等能力；
- **socket 栈**：基于 smoltcp 支持 TCP/UDP，并实现 Unix domain socket 和常见 socket syscall；
- 等待与同步：实现 futex 基础等待/唤醒、robust list 相关路径，以及 pipe/eventfd/epoll 的阻塞和唤醒机制；
- 可选调度策略：默认按 nice 加权 vruntime 运行简化 CFS，并可在编译期切换到 FIFO RR；
- 双架构适配：同一内核代码支持 RISC-V64 与 LoongArch64，设备侧分别适配 MMIO 与 PCI VirtIO。

11.1.4 Rust 工程实践

内核大量使用 Arc、Weak、Mutex、RwLock、trait 和 enum 分发管理复杂对象生命周期。Rust 的类型系统降低了普通内存错误概率，但内核仍需要在 FFI、页表、DMA、用户态指针和自引用结构中使用 unsafe，这些区域是后续审计和封装的重点。

11.2 项目特色

1. 真实 **ext4** 后端：相比只使用教学文件系统，Ya2yOS 直接在 lwext4 之上构建 VFS 适配，能够挂载和读写 ext4 镜像中的真实用户态文件。
2. 兼容性优先的 **syscall** 策略：对 glibc 和 LTP 高频路径持续补齐语义，同时对 io_uring_setup、signalfd4、memfd_create 等探测型接口提供可控 stub，避免用户态过早失败。
3. 文件描述符统一模型：普通文件、socket、pipe、eventfd、epoll、inotify、mqueue、设备文件和挂载上下文都通过 fd 表统一管理。

4. 双架构和双 **VirtIO** 传输：RISC-V64 使用 MMIO VirtIO，LoongArch64 使用 PCI VirtIO，验证了内核架构抽象和设备层解耦。
5. 面向测试驱动的演进：大量修复围绕 libc-test、LTP、BusyBox 和实际日志展开，文档和 problem 记录保留了问题定位与修复路径。

11.3 与同类项目的对比

特性	Ya2yOS	xv6	rCore	ArceOS
实现语言	Rust	C	Rust	Rust
架构	RISC-V64 + LoongArch64	RISC-V64	RISC-V64	多架构
内核形态	宏内核	宏内核	宏内核/教学	组件化/unikernel
文件系统	ext4 (lwext4) + 兼容 dev/proc	简单 FS	easyfs/FAT 类	多组件 FS
网络	smoltcp TCP/UDP + Unix socket	无	smoltcp 子集	smoltcp
用户态目标	glibc/musl/BusyBox/LTP 子集	自定义用户态	教学用户态	定制应用
设备	VirtIO blk/net，MMIO/PCI	简单设备	VirtIO 子集	VirtIO 多设备

11.4 已知局限

1. 页缓存写回深化：文件页缓存已覆盖 read/mmap/splice 并共享物理页，但脏页回写仍由 lwext4 与 sync 路径完成，缓存模块自身不承担回写调度。
2. 挂载语义仍受路径式 **VFS** 限制：挂载表已处理叠加层、bind/move 子树及 shared/slave 传播，但尚未实现独立 superblock、挂载点 dentry 切换和 mount namespace；当前 bind 可见性通过目录镜像近似。
3. 网络轮询驱动：TCP/UDP 依赖 poll_interfaces() 周期性推进，VirtIO-net 中断路径尚未完整接管收包和唤醒。
4. 部分 **syscall** 为兼容 **stub**：如 io_uring_setup (仅占位 fd 与 ring offsets ABI)、timerfd_create、memfd_secret、perf_event_open 及 fanotify 权限事件响应等尚未提供完整语义。
5. 权限和安全模型有限：已有 uid/gid、mode、umask 和部分访问检查，但 capabilities、seccomp、namespace、LSM 等机制仍缺失。
6. 调度和多核能力有限：已有多 Hart processor、远程入队、IPI/空闲 Hart 通知及 100Hz 抢占，并可编译期选择 CFS/RR；但尚无完整跨 Hart 迁移、work stealing、负载均衡、NUMA 感知、实时类和完整 Linux SCHED_* 运行时策略。
7. 设备模型不完整：/dev 主要是手工注册兼容层，loop 设备尚无真实 backing file 数据路径，图形/输入/熵设备未系统接入。

11.5 未来发展方向

11.5.1 短期目标

1. 页缓存与文件一致性：在现有 read/mmap/splice 共享缓存页基础上，完善脏页回写调度与缓存容量策略。
2. 网络中断与唤醒：完善 VirtIO-net 中断、socket waker 和 epoll 唤醒路径，降低轮询延迟和 CPU 消耗。
3. 真实 **loop** 设备：把 loop 读写转发到 backing file，支持镜像挂载和更多 LTP 文件系统测试。
4. **stub** 梳理：对当前兼容 stub 分类，明确哪些应返回能力不足错误，哪些需要补真实语义。

11.5.2 中期目标

5. **tmpfs/procfs/devtmpfs** : 将 /proc、/dev、/tmp 从 ext4 承载的兼容文件提升为独立虚拟文件系统。
6. 完整挂载树 : 在现有传播状态和 event group 之上实现挂载点 dentry 切换、多 superblock、mount namespace，并将新挂载 API 接到真实 VFS 对象。
7. 调度与多核 : 完善 CFS 调度周期与唤醒放置，实现 CPU 亲和性迁移、跨 Hart 负载均衡和实时调度类。
8. **io_uring/AIO** : 在页缓存和设备 waker 基础上实现高性能异步 IO。

11.5.3 长期方向

9. 容器相关机制 : 逐步补齐 namespace、cgroup、capabilities 和 seccomp，为容器运行时打基础。
10. 更多设备与图形 : 接入 virtio-rng、virtio-input、virtio-gpu，支持基础图形输出和更完整的设备发现。
11. 更强网络能力 : 扩展 IPv6、netlink、raw/packet socket、路由配置和网络诊断接口。
12. 工程化验证 : 扩大自动化测试矩阵，覆盖 RISC-V64/LoongArch64、musl/glibc、libc-test/LTP/BusyBox 等组合。

11.6 工程实践反思

11.6.1 Rust 在内核中的收益与代价

Rust 的所有权系统让内核对象生命周期更明确，尤其适合 fd、socket、inode、task 等引用关系复杂的对象。但内核开发无法完全避免 unsafe：页表地址转换、用户指针拷贝、DMA、FFI 和中断上下文都需要人工维护不变量。因此，Rust 在这里提供的是更强的默认安全边界，而不是免除内核工程纪律。

11.6.2 调试方式

当前调试主要依赖：

- 分级日志和定向日志文件；
- QEMU/GDB；
- panic backtrace；
- 用户态回归测试；
- 对 LTP/libc-test 失败日志做最小复现；
- problem 文档记录根因和修复方案。

11.6.3 后续维护建议

1. 修改 syscall 前先确认 Linux 错误码、边界条件和用户指针访问规则。
2. 修改内存、文件、任务等共享路径时优先补回归测试。
3. 对兼容 stub 保持诚实标注，避免上层误判能力已经完整。
4. 遇到跨架构问题时同时检查地址布局、页表权限、DMA 地址和 ABI 结构布局。
5. 文档应随功能边界一起更新，尤其是“已实现”和“兼容返回”的区别。

11.7 结语

Ya2yOS 已经从教学内核雏形推进到可以运行真实 Linux 用户态负载的实验系统。它仍有许多工程债和语义缺口，但核心路径已经成形：进程、内存、文件、网络、信号和设备能够围绕 Linux ABI 协同工作。

后续工作的重点不只是增加 syscall 数量，而是把已有兼容面做深：减少 stub、完善缓存和挂载语义、提升网络与 IO 的事件驱动能力，并用持续测试保持双架构行为一致。

第十二章 实现追溯与参考资料

12.1 源码—章节索引

主题	主要实现位置	本文位置
内核入口与初始化	os/src/main.rs	第 2 章
架构实现	os/src/arch/riscv64/、os/src/arch/loongarch64/	第 2 章
进程、调度与 futex	os/src/task/、os/src/task/scheduler/、os/src/timer/	第 3 章
页表、VMA 与用户复制	os/src/mm/、os/src/sync/remote_tlb.rs	第 4 章
信号动作、pending、frame 与 trampoline	os/src/signal/、os/src/syscall/signal.rs、os/src/arch/*/qemu/	第 5 章
syscall ABI 与实现分发	os/src/syscall/ (fs/mm/task/net/ipc/io_mpx/sys/sync)	第 3-8 章
socket 和协议栈封装	os/src/net/、os/src/drivers/net/	第 6-7 章
VirtIO、IRQ 与平台设备	os/src/drivers/virtio/、os/src/arch/irq/	第 7 章
VFS、ext4、proc、pipe 与挂载	os/src/fs/、os/src/drivers/disk.rs	第 8 章
lwext4 磁盘文件系统引擎	crates/lwext4_rust/、os/src/fs/ext4_lw/	第 9 章
时间、同步与性能诊断	os/src/timer/、os/src/sync/、os/src/utils/、os/src/utils/perf/	第 3-5、11 章

12.2 参考资料

- [1] 《The RISC-V Instruction Set Manual, Volume II: Privileged Architecture》. 2024 年. [在线]. 载于: <https://riscv.org/technical/specifications/>
- [2] 《LoongArch Architecture Reference Manual, Volume 1: Basic Architecture》. 2023 年. [在线]. 载于: <https://www.loongson.cn/product/show?id=8>
- [3] 《The Rust Programming Language》. [在线]. 载于: <https://doc.rust-lang.org/book/>
- [4] 《Linux man-pages project》. [在线]. 载于: <https://www.kernel.org/doc/man-pages/>
- [5] 《smoltcp: a standalone, event-driven TCP/IP stack》. [在线]. 载于: <https://github.com/smoltcp-rs/smoltcp>